
ELEC40006

Project Report

Group 10

Christopher Koh (02589209)
Devesh Khemani (02723399)
Albert Ma (02679083)
Wangmo Koo (02577935)
Zifan Ye (02669492)

Word Count: 75,90

Contents

- . **1. Introduction**
- . **2. Requirements**
- . **3. Planning**
- . **4. System Design**
- . **5. Sensor Design**
 - . **5.1 Magnetic Detection**
 - . **5.2 Infrared Detection**
 - . **5.3 Ultrasound Detection**
 - . **5.4 Radio Detection**
- . **6. Robot Design**
 - . **6.1 Movement**
 - . **6.2 User Interface**
- . **7. Complete Rover**
 - . **7.1 Integration**
 - . **7.2 Testing**
- . **8. Conclusion**

1. Introduction

This report covers the design, implementation and testing of the EEE Lunar Rover project completed by Group 10. The objective of this project was to build a remotely controllable lunar rover capable of identifying different types of lunar rocks. The rover was required to detect and classify rocks based on radio, infrared, ultrasound and magnetic signals. This was accomplished using multiple sensors and an in-built control system.

Besides the core objectives of the project, additional considerations made during the design process included maintaining a rover weight below 750g, design cost and effectiveness, rover manoeuvrability and construction, and remote control interface design.

As detailed in this report, electronic detection subsystems were designed and integrated with a Metro M0 Express microcontroller to detect the necessary signals. Preliminary circuit design before experimental testing was conducted to implement these subsystems. Appropriate chassis design and wifi interface webpage software were implemented as well, to create a fully working rover.

2. Requirements

The objective of the project is to enable the rover to identify different types of lunar rocks. Each lunar rock can be characterised depending on radio, infrared, ultrasound and magnetic signals.

The rock's age is encoded as ASCII characters and framed as UART packets in the radio signal. The radio signal has a carrier frequency of 89kHz and is modulated with two-level amplitude-shift-keying, where an amplitude of 0 represents logic 0 and an amplitude greater than 0 represents logic 1. Data speed is 600 bits per second. Each rock's age is four characters long, and starts with a “#” symbol. For example, #123 means the rock's age is 1.23 billion.

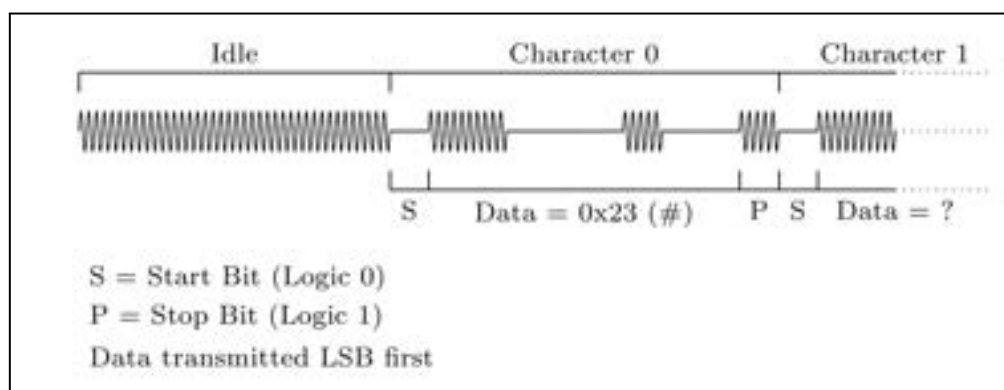


Figure 2.1. Diagram showing radiowave age transmission.

To determine the type of rock, infrared, ultrasonic and magnetic signals are considered, as shown in Figure 2.2. The rock emits an infrared signal of wavelength 950nm randomly in time with a Poisson distribution. The ultrasound signal emitted has a frequency of 40kHz. The magnetic field of the rock is static and has a direction either up or down.

Types	Infrared	Ultrasound	Magnetic
Basaltoid	$\lambda=547 \text{ s}^{-1}$	40kHz	Down
Gravion	$\lambda=312 \text{ s}^{-1}$		Down
Regolix	$\lambda=312 \text{ s}^{-1}$	40kHz	Up
Lunarite	$\lambda=547 \text{ s}^{-1}$		Up

Figure 2.2. Rock types and their characteristics.

Besides identifying the age and type of the rock, the rover must fulfil a weight requirement. The rover's capability will be assessed in a demo environment with obstacles and rocks spaced throughout the demo surface. The weight of the rover must not exceed 750g, so that it can traverse the "fragile surface" area on the demo environment.

The rover design must also bear in mind cost and weight effectiveness, sufficient manoeuvrability to negotiate the environment, a robust and reliable construction, and a logical and intuitive user control interface.

3. Planning

Planning was critical to ensure the project was completed smoothly, and began with an assessment of the expansion kit provided for the project. The expansion kit contained a motor driver module to assist in motor power control. An Adafruit Metro M0 Express board and an Adafruit WINC1500 WiFi Shield were also included to replace the existing EEEBug Arduino board.

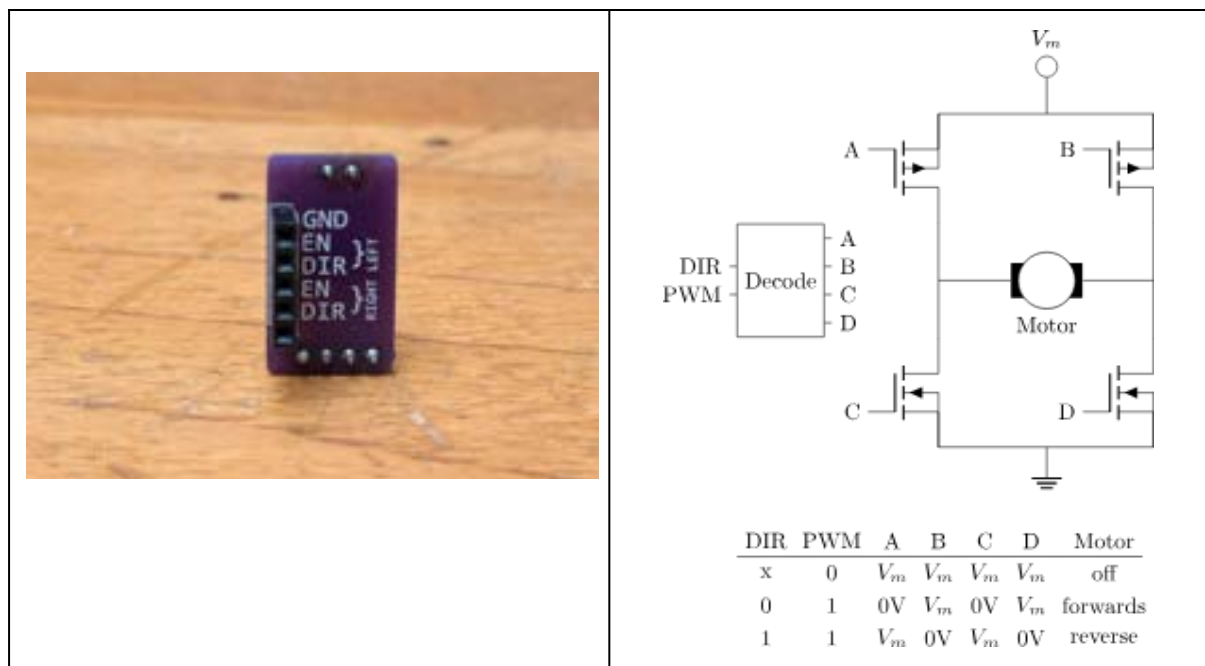


Figure 3.1. Motor driver module (left) and its decode logic (right).

The motor driver module contains two H-Bridges, one for each motor, with decode logic shown above. It uses four MOSFETs configured as electronic switches to control the motors. The DIR and PWM input signals are used to determine direction and motor speed and can be connected directly to the Metro board's digital outputs. The motor driver module plugs into the motor power connectors on the EEEBug PCB, enabling motor control.

The Adafruit Metro M0 Express board is an embedded computing platform based on an ARM M0 microcontroller and compatible with the Arduino form factor, libraries and development tools. It is programmed in C++ and features digital inputs and outputs, inputs with digital-to-analogue conversion and PWM outputs. The Adafruit WINC1500 wifi shield provides internet connectivity and plugs into the Metro board. The Metro board can be powered using the EEEBug batteries by connecting its 5V and GND pins to the EEEBug PCB 5V and GND pins, respectively.

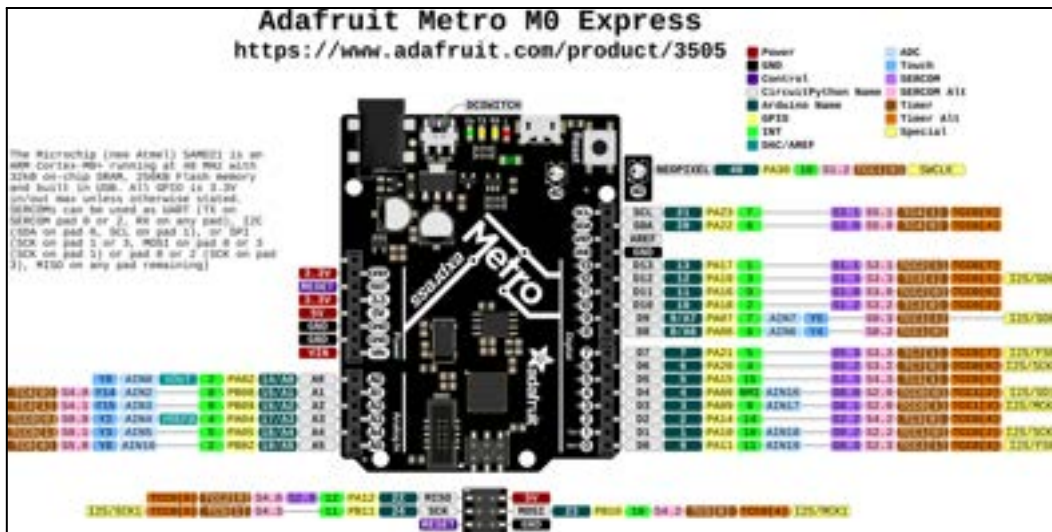


Figure 3.2. Adafruit Metro M0 Express board.



Figure 3.3. Adafruit WINC1500 Wifi Shield.

An EEE Rover network was also set up in the lab. By connecting the wifi shield to the Metro board, the Metro microcontroller can connect to the EEE Rover network. This enables webpage hosting to implement a rover control interface. The expansion kit thus enabled a conversion of the EEE Bug chassis into a suitable rover base.

Considering the requirements and the expansion kit, it was identified that the project workload could be split into two categories: robot design and sensor design. Sensor design comprised four sensors: radio, infrared, ultrasound and magnetic. Robot design involved reconfiguring the EEE Bug chassis and developing a webpage to enable user control of the rover. The project workload was divided into six sections, as shown in Figure 3.4.

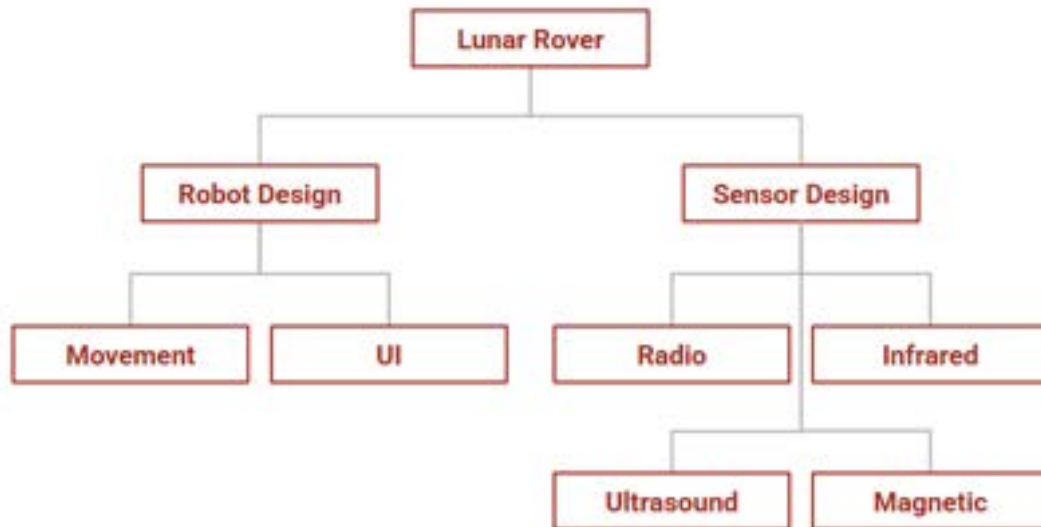


Figure 3.4. Design hierarchy of the lunar rover.

Dividing the workload this way enabled simultaneous work on the robot design and sensor design, maximising efficiency. Each group member was assigned one design component. Each member performed their own research on how to implement their assigned project component and submitted their list of required parts to Wangmo, the group Parts Approver, so that they could execute their designs.

Group Member	Role
Christopher Koh	Project Manager + Infrared Detection + Integration
Devesh Khemani	Magnetic Detection
Zifan Ye	Radio Detection
Wangmo Koo	Ultrasound Detection + Parts Approver
Albert Ma	Movement & User Interface

Figure 3.5. Project role allocation.

The project timeline was outlined as shown in the Gantt chart below.



Figure 3.6. Gantt chart of project workflow.

4. System Design

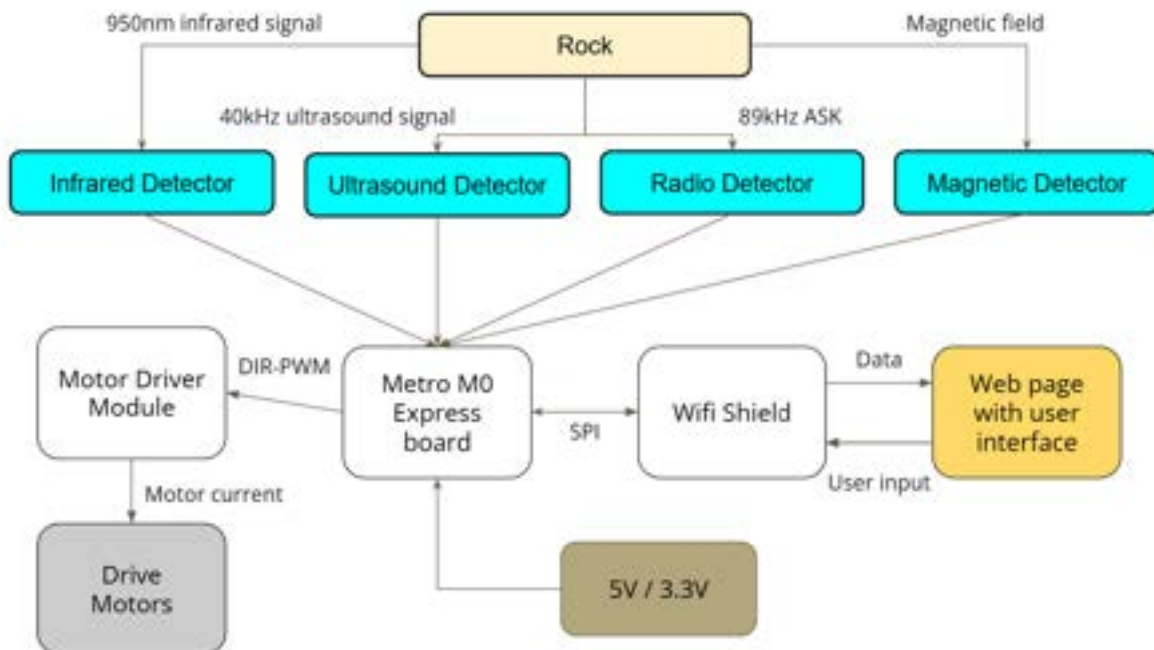


Figure 4.1. Overall system design infrastructure diagram.

The chosen system design is shown in Figure 3.1. Four detector subsystems (infrared, ultrasound, radio and magnetic) are implemented to receive the corresponding signals from the rock. Each subsystem then sends the relevant data, via either an analogue or digital signal, directly to the Metro M0 Express board, which acts as the rover's central interface.

The Metro board then interprets the data received and communicates that to the WINC1500 wifi shield, using SPI communication. The wifi shield then wirelessly sends the data to the web page that the user can access from their device. Similarly, user inputs from the web page are transmitted to the Wi-Fi shield, which communicates them to the Metro board via SPI.

User inputs for movement are sent from the Metro board to the motor driver modules to control the drive motors. The rover batteries provide power for the motors and the sensor subsystems, with power being regulated at 3.3V and 5V by the Metro board's in-built voltage regulators.

5. Sensor Design

This section of the report covers in-depth explanations of the design choices, testing and final implementation of the following sensor subsystems:

1. **Magnetic Detection**
 2. **Infrared Detection**
 3. **Ultrasound Detection**
 4. **Radio Detection**
-

5.1. Magnetic Detection

Background

The rocks have their own magnetic field that points either up or down. Basaltoid and Gravion have a downwards-pointing magnetic field, whereas Regolix and Lunarite have an upwards-pointing magnetic field.

Considerations

The magnetic field of the rock is static, so it will only induce a current in a coil of wire if the coil is moving. The rover motors also have a magnetic field that can potentially interfere with the detection circuit. The Metro board inputs cannot receive voltages greater than 3.3V.

Preliminary Design

To detect the direction of the rock's magnetic field, two approaches were considered. One, using a moving coil to induce a current and then determining the direction of the field based on the current direction. Two, using a hall sensor to determine the field direction directly.

A Hall-Effect sensor works by detecting changes in a magnetic field using a principle called the Hall effect. When an electrical current flows through a conductive or semiconducting plate, and a magnetic field is applied perpendicular to that current, the magnetic force pushes the moving electrons toward one side of the plate. This unequal distribution of electrons creates a voltage difference across the sides of the plate. This voltage is directly proportional to the strength of the magnetic field, enabling the sensor to determine the direction and strength of a nearby magnet.

Ultimately, a Hall effect sensor was chosen over a moving coil approach. Since the rocks are stationary, maintaining a moving coil to determine the magnetic field is complicated to implement. In contrast, the Hall sensor directly measures the magnitude and polarity of a magnetic field without requiring motion. Hall sensors also produce a voltage that can be easily read by the Metro board as an analogue input. It is more reliable and simpler to implement.

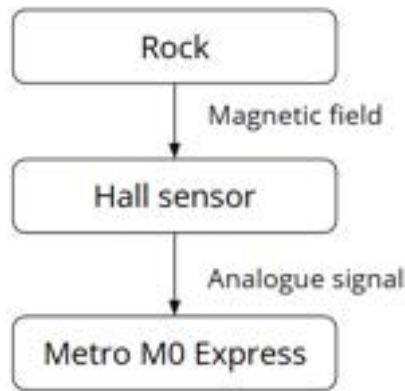


Figure 5.1.1. Magnetic subsystem detection flow.

The hall sensor chosen was the DRV5053RAQLPG, for its compatibility with the Metro board power supply, its sensitivity and its output range. The DRV5053RAQLPG outputs a continuous voltage between 0.2V and 1.8V. When no magnetic field is detected, the voltage hovers at 1V. When an upward field is detected, the voltage rises to 1.8V, and when a downward field is detected, the voltage falls to 0.2V. The maximum output does not exceed 3.3V and will not damage the Metro board when fed to its analogue pin.

DRV5053RAQLPG	Characteristics
	• 2.7–38V supply, works directly on Metro M0 3.3V • Output 0.2–1.8V • Sensitivity –45 mV/mT • Saturates at 18 mT, fine for polarity detection • 20 kHz bandwidth, chopper-stabilised for low noise offset • Reverse supply, overcurrent and load dump protection • Operating range –40 to 125°C

The voltage output of the hall sensor is then converted by the Metro board to a number from 0-1023 (ADC), where the analogue number = $\frac{\text{Voltage output}}{3.3} \times 1023$. When the output number is 310, there is no magnetic field detected. When the output number is less than 310, the hall sensor is detecting a south pole. If the output number is greater than 310, a north pole is being detected.

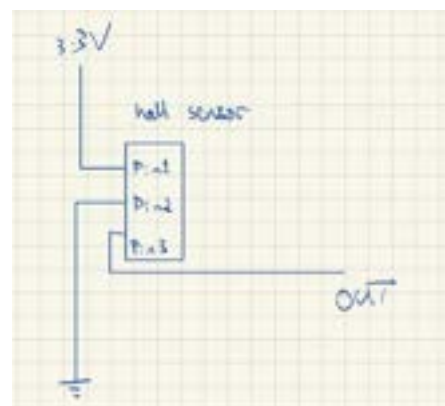


Figure 5.1.2. Hall sensor circuit sketch.

Testing

During testing, it was found that the hall sensor was weak and only picked up the magnetic field from a distance of approximately 2cm from the rock. This was likely due to the rock casing blocking the magnetic field of the magnet placed inside. To solve this, an opamp was used to amplify the signal from the sensor.

An MCP6022 opamp was used to amplify the signal. The gain of the opamp was configured to a gain of 2, effectively increasing the output range to 0.4-3.6V. While the 3.6V maximum output of the hall sensor exceeded the 3.3V maximum input of the Metro board, this was deemed not an issue, as the final 0.3V of the upper output range would only be measured at a very close distance to the magnet, since the magnetic field strength follows an inverse cube relationship with distance. Any gain greater than 2 would far exceed the maximum reading.

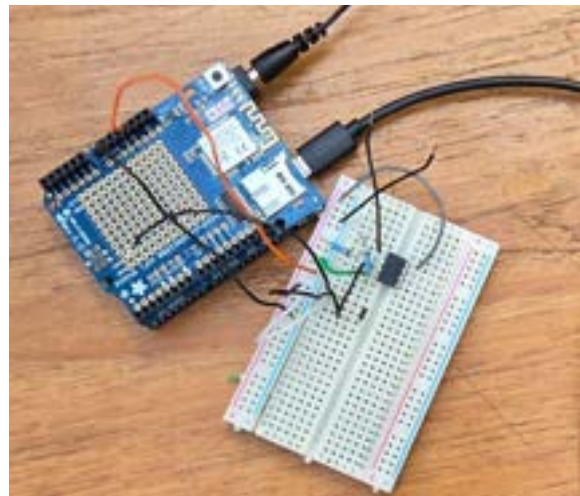
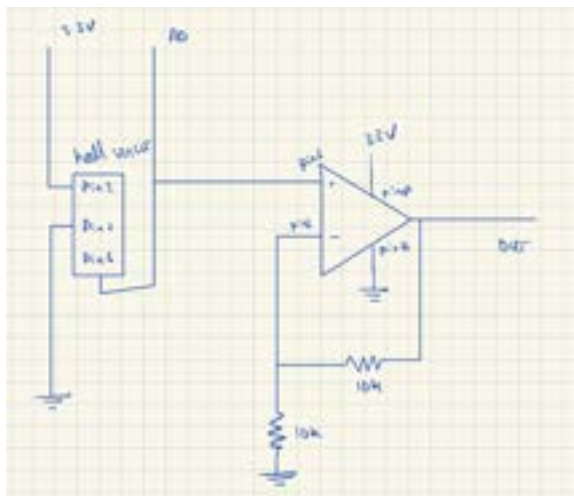
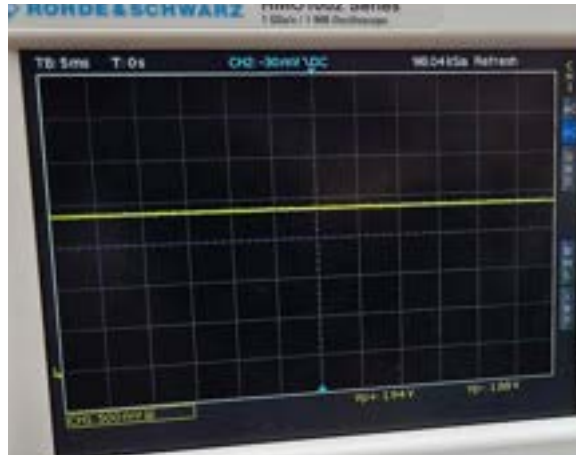


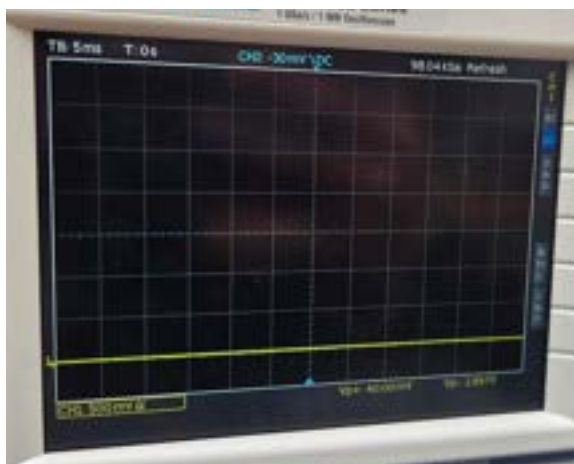
Figure 5.1.3. Circuit sketch with opamp.

Figure 5.1.4. Magnetic detection circuit on breadboard.

With the opamp introduced to the circuit, the following test results were obtained using an oscilloscope to measure the output voltage of the hall sensor. The voltage output by the hall sensor hovered at roughly 1.94V when no magnetic field was present. When a downwards magnetic field was present, the voltage dropped to 0.4V as expected. When an upwards magnetic field was present, the voltage rose to roughly 3.3V. It was noted that the flat side of the sensor was pointing down.



No magnet present



South pole present



North pole present

Figure 5.1.5. Oscilloscope readings of magnetic detection circuit output.

Final Implementation

To complete the implementation of the magnetic detection circuit, the output of the circuit was connected to analogue pin 4 of the Metro board. The following code was written to convert the output into a user-readable value. With the amplification circuit active, the baseline quiescent voltage shifted to approximately 1.94V, a theoretical ADC value of 601. Upon testing, this ADC value was determined as 610 instead, due to hardware tolerances and resistor variances.

To prevent sensor noise or minor environmental fluctuations from triggering false positives, a software deadband (buffer) of ± 10 ADC counts was implemented around the 610 baseline. This relatively small buffer value was selected because the rock material made the magnet's magnetic field slightly harder to read, and registering a change in the field strength was sufficient to determine the direction of the magnetic field.

```
void setup() {  
  pinMode(A4, INPUT);  
  Serial.begin(9600);  
}  
  
void loop() {  
  int MagVal = analogRead(A4);  
  
  if (MagVal > 620) {  
    Serial.println("Magnetic Field: UP (North)");  
  } else if (MagVal < 600) {  
    Serial.println("Magnetic Field: DOWN (South)");  
  } else {  
    Serial.println("No strong magnetic field detected.");  
  }  
  
  delay(200);  
}
```

Figure 5.1.6. Arduino code used to test magnetic detection circuit.

5.2. Infrared Detection

Background

The rocks emit an infrared signal that has a pulse rate with a Poisson distribution. There are two pulse rates to distinguish, approximately 312s^{-1} and 547s^{-1} . Each pulse is roughly $50\mu\text{s}$ wide, and the wavelength of the infrared rock emission is 950nm .

Considerations

The infrared signal output from the rock experiences interference from ambient light. The ambient light in the testing area comprises mainly electric light, which has a strong frequency component of 100Hz . Noise will be introduced into the signal. Additionally, the infrared signal is emitted from within the rock. The rock casing that covers the internal emitter will block the signal and make the signal output weak, so amplification is required.

The Metro board inputs cannot receive voltages greater than 3.3V .

Preliminary Design

Infrared signals can be received using a phototransistor. A phototransistor is a semiconductor device that converts light into current. It acts like a light-dependent current source and is equally sensitive to infrared radiation as visible light. When an infrared signal is incident on the phototransistor, it generates a current that can then be passed through a resistor to generate a voltage. The voltage generated can then be sent to the Metro M0 Express board to register a pulse. Depending on the number of pulses detected within a finite time, the pulse rate can be calculated.

Since the infrared signal is weak, the current generated will be small, and consequently, the voltage dropped across the resistor will be small. The output from the phototransistor will need to be amplified. Additionally, since pulses need to be counted, a comparator can be used to convert the analogue signal from the infrared pulses into a digital signal for cleaner readings. To remove noise, filters can be implemented where necessary. The detection flow is shown in Figure 5.2.1.

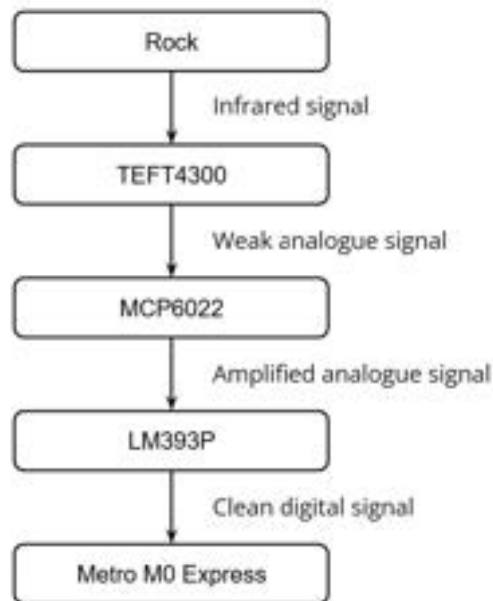


Figure 5.2.1. Infrared subsystem detection flow.

The following parts were selected to execute the infrared detection circuit.

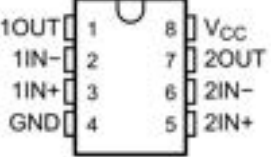
Part	Purpose	Characteristics
TEFT4300 	Phototransistor to detect infrared pulses	- Has a daylight blocking filter matched with 940nm - Peak sensitivity: 925nm - Total rise and fall time: 11μs
MCP6022 I/P 	Opamp to amplify the output from TEFT4300	- Power supply range: 2.5-5.5V - Gain-bandwidth product: 10MHz - Slew rate: 7V/μs - Dual channel
LM393P 	Comparator to convert the output from the MCP6022 into a clean digital signal that the Metro can read	- Open-drain NPN transistor output - Single supply compatible: 2-36V - Response time: ~1.5μs

Figure 5.2.2. Table of parts selected for infrared detection.

The TEFT4300's daylight blocking filter is an analogue filter to reduce interference from the ambient light. Its peak sensitivity at 925nm ensures it can pick up the infrared signal of 950nm. Its response time is 11 μ s, which is sufficiently smaller than the 50 μ s pulse width, so it can respond to the pulses.

The MCP6022 can operate on the 3.3V and 5V power supplies from the Metro board. Its 10MHz gain-bandwidth product and 7V/ μ s slew rate make it capable of amplifying the fast infrared pulses without excessive waveform distortion. It is also dual-channel, so a second gain stage or filter can be implemented while simplifying the circuit.

The LM393P can operate from the 3.3V and 5V power supplies and has a response time of around 1.5 μ s, so it can detect the 50 μ s infrared pulses without significant timing distortion. The output requires a pull-up resistor, so the output can be capped at 3.3V so that the Metro board is not damaged.

Hence, the TEFT4300, MCP6022 and LM393P are all well-suited for use in the infrared detection circuit.

Circuit

The proposed infrared detection circuit was sketched in LTspice before construction on a breadboard.

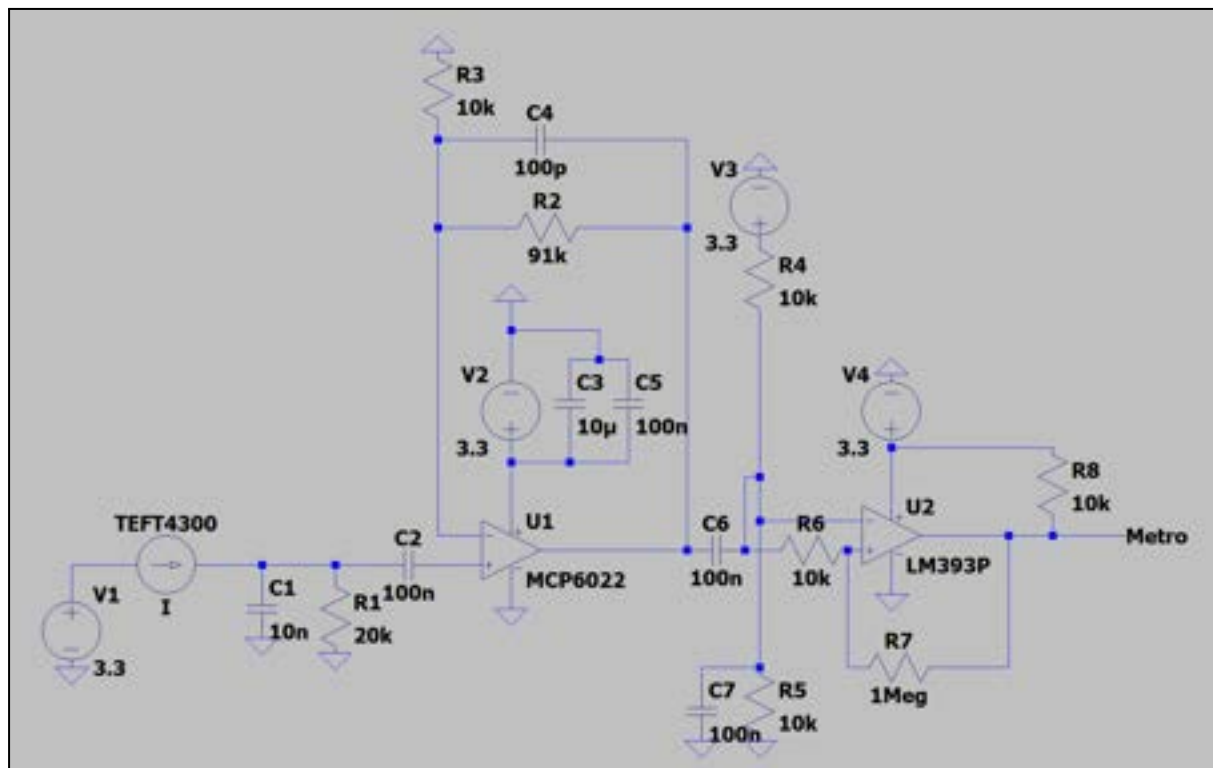


Figure 5.2.3. LTspice sketch of infrared detection circuit.

The TEFT4300 is modelled as a current source. R1, the load resistor, converts the phototransistor current into a voltage. Together with C1, they form a low-pass filter with a cut-off frequency of $\frac{1}{2\pi(20k)(10n)} \approx 796Hz$, to remove high-frequency noise. C2 removes DC offsets from ambient light before amplification.

The amplifier network is non-inverting and uses resistors R2 and R3 to set up a gain of 10.1. C4 is connected in parallel with R2 to reduce high-frequency gain, improving amplifier stability by preventing oscillation and excessive noise amplification. C3 and C5 (10 μ F electrolytic and 100nF ceramic) are power supply decoupling capacitors, respectively providing low-frequency supply smoothing and suppressing high-frequency transients and noise on the supply rail.

C6 and R6 form a high-pass filter with a cut-off frequency of $\frac{1}{2\pi(10k)(100n)} \approx 159Hz$ to remove the 100Hz interference component introduced by the ambient electric light sources. R4 and R5 form a voltage divider to create a reference voltage of 1.65V, while C7 filters noise on this reference voltage to improve comparator stability. R7 provides positive feedback to prevent false triggering due to noise near the switching threshold, improving pulse detection reliability. Finally, R8 is the pull-up resistor for the LM393P open-collector output, pulling the output to the 3.3V supply rail, producing a valid digital HIGH level for the Metro input.

The final complete circuit was then built on a breadboard, as shown in Figure 5.2.3, for testing.

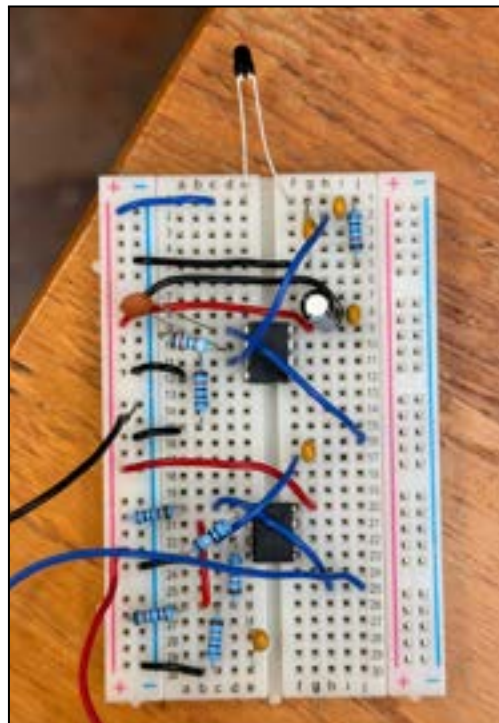


Figure 5.2.3. Breadboard infrared detection circuit.

Testing

Testing was conducted using an oscilloscope. The channel 1 probe was connected to the phototransistor output, while the channel 2 probe was connected to the comparator output. The circuit was tested with the rock simulator set to emit the infrared signal with a 312s^{-1} pulse rate and the 547s^{-1} signal.

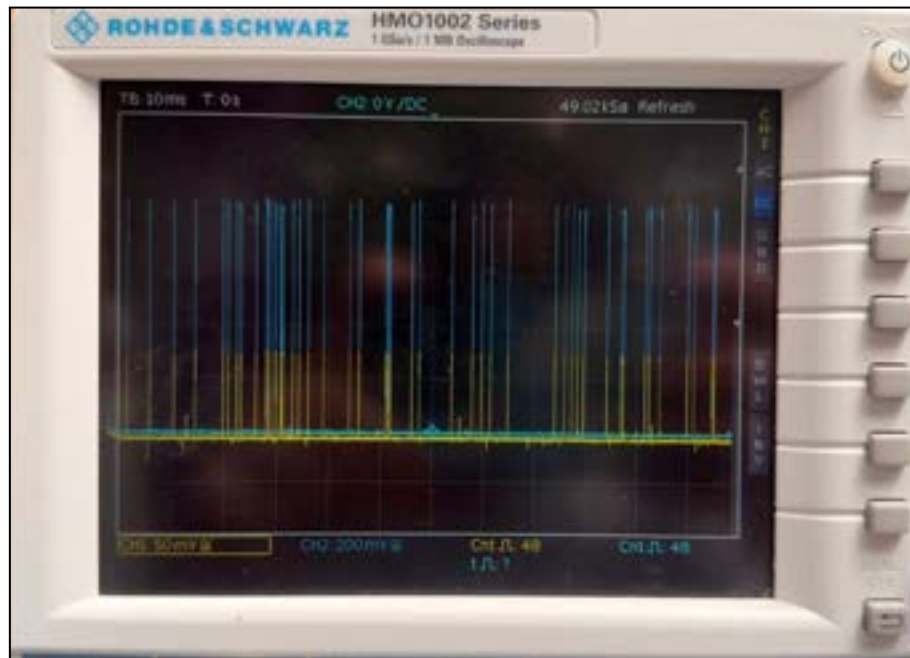


Figure 5.2.4. Oscilloscope reading for infrared 312s^{-1} test.



Figure 5.2.5. Oscilloscope reading for infrared 547s^{-1} test.

The infrared signal was correctly detected as sharp pulses. For both signal pulse rates, a small and noisy phototransistor output (CH1, yellow measurement)

was measured, and an amplified, clean comparator output (CH2, blue measurement) was measured. Using the auto-measure function on the oscilloscope to count the number of pulses, around 40 pulses were counted for the 312s^{-1} pulse rate and around 60 pulses were counted for the 547s^{-1} pulse rate.

Final Implementation

To complete the implementation, the comparator output was connected to digital pin 2 of the Metro board. The digital input pin receives either 0V or 3.3V from the comparator, which is interpreted by the Metro as binary 0 and 1, respectively. To determine the pulse rate, the number of pulses within a given time is counted and then divided by the elapsed time.

```
1  int pulseCnt = 0;
2  long lastIRTime = 0;
3  float IRpulseRate;
4
5  void countPulse() {
6      pulseCnt++;
7  }
8
9  void setup() {
10     pinMode(2, INPUT);
11     attachInterrupt(2, countPulse, RISING);
12     Serial.begin(9600);
13 }
14
15 void loop() {
16     long elapsedTime = millis() - lastIRTime;
17     if (elapsedTime > 200) {
18         lastIRTime = millis();
19         IRpulseRate = (float) pulseCnt * 1000 / (float) elapsedTime;
20         pulseCnt = 0;
21     }
22     if(IRpulseRate > 400){
23         Serial.println("547s-1: Basaltoid OR Lunarite");
24     }
25     if((IRpulseRate > 240 ) && (IRpulseRate < 370)){
26         Serial.println("312s-1: Gravion OR Regolix");
27     }
28     if(IRpulseRate < 240){
29         Serial.println("No IR detected");
30     }
31 }
```

Figure 5.2.6. Arduino code used for Metro to interpret infrared signals.

The code sets up an interrupt that increments the pulse count by one every time a rising edge is detected from pin 2, as this corresponds to a HIGH. Using an internal timer, the code then calculates the pulse rate every 500ms.

A time interval of 500ms was chosen as the infrared pulses follow a Poisson distribution, where $\sigma = \sqrt{N}$ (σ is the standard deviation and N is the number of counts). During testing, over 120ms, approximately 40 and 60 counts were observed for the pulse rates of 312s^{-1} and 547s^{-1} , respectively. Measuring the pulse rate over 500ms obtained $N = 167$ and $N = 250$, returning a fractional uncertainty of 7.7% and 6.3% respectively, so a statistically relevant result could be calculated.

Then, depending on the calculated infrared pulse rate, either 312s^{-1} or 547s^{-1} is displayed. A buffer region was included to ensure the Metro can correctly distinguish between the two pulse rates, with some margin for error. If the pulse rate calculated is not significant, "No IR detected" is displayed.

A final test was conducted, now with the appropriate arduino code uploaded to the Metro. The following results were obtained.

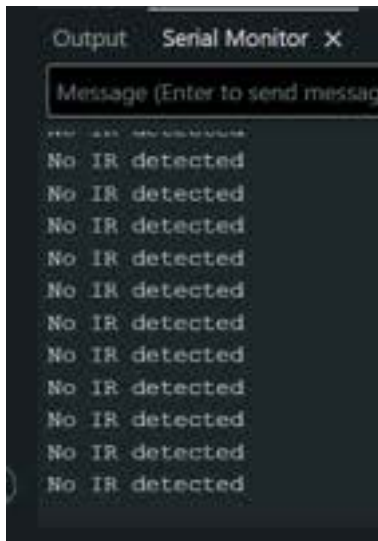
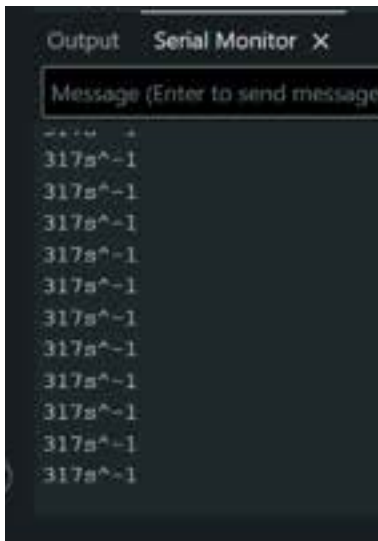
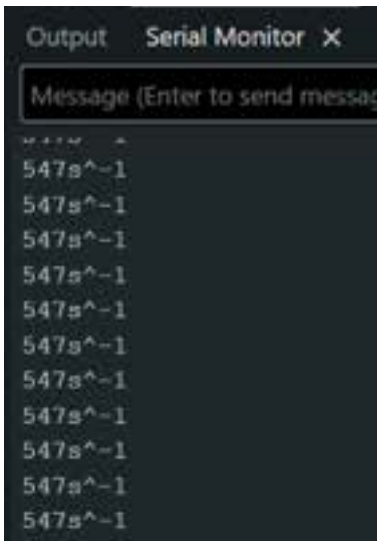
No IR	312s^{-1}	547s^{-1}
		

Figure 5.2.7. Serial monitor outputs from final testing.

5.3. Ultrasound Detection

Background

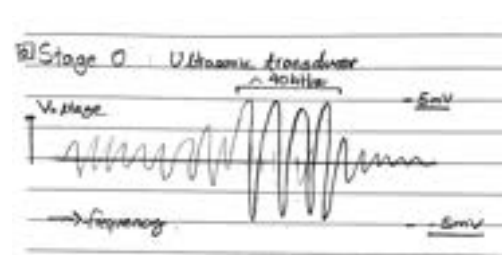
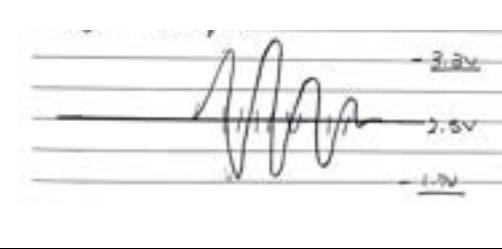
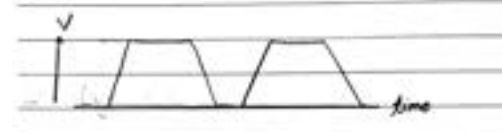
The rock emits a 40kHz ultrasound signal depending on its rock type. The rover needs to determine if the signal is present or absent.

Considerations

The ultrasound signal might experience interference and noise from motor vibrations, ambient acoustic noise, etc. The ultrasound signal will be weak due to the rock casing, so amplification is necessary. The Metro board inputs cannot receive voltages greater than 3.3V.

Preliminary Design

An ultrasonic transducer can be used to receive the ultrasound signal. The ultrasound signal will have a sinusoidal waveform, which can be converted as shown below:

	<u>Ultrasonic Transducer</u> Receives a 40 kHz AV signal.
	<u>Amplifier + Filter</u> Amplifies the signal with gain and limits the bandwidth to around 40kHz.
	<u>Envelope Detector</u> Rectifies and smoothes the high-frequency AC signal to a slow DC envelope.

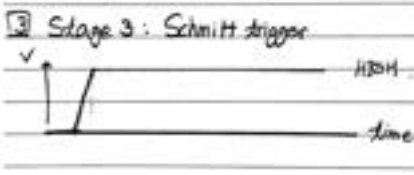
	Schmitt-trigger Comparator Schmitt Trigger converts those pulses into a clean digital signal. With a high voltage outlining the presence and 0V representing the absence.
---	--

Figure 5.3.1. Flowchart of ultrasound signal transformation.

The detection flow is as follows to convert the ultrasound signal into a pulse that can be easily detected by the Metro board.

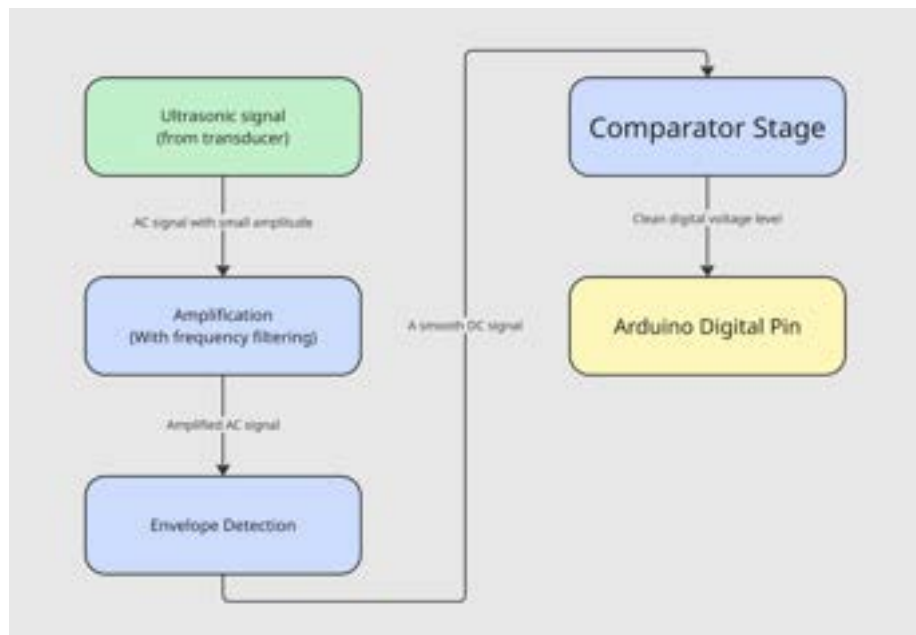


Figure 5.3.2. Ultrasound subsystem detection flow.

To execute this, the following parts were selected:

Part	Purpose	Characteristics
400SR10B 	Ultrasonic transducer to receive the ultrasound signal	- Centered at 40kHz receiver with narrow bandwidth (+/- 1~2 kHz) - 2.5 ~ 38V power supply - Receiving sensitivity : - 40.0Khz 0dB = 1 volt/μbar

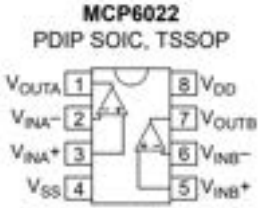
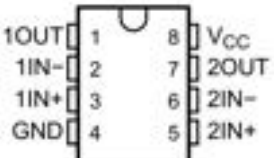
MCP6022 I/P 	Opamp to amplify the output from the 400SR10B	- Power supply range: 2.5-5.5V - Gain-bandwidth product: 10MHz (since the target frequency is 40kHz, a maximum gain of 10M/40k = 250 is possible) - Slew rate: 7V/μs - Dual channel
1N4148 	Diode used for the envelope detector	- Fast-switching; 5ns reverse Recovery - Forward voltage max: 1V - Operating Temperature Max: 175 °C
LM393P 	Comparator to convert the output from the MCP6022 into a clean digital signal that the Metro can read	- Open-drain NPN transistor output - Single supply compatible: 2-36V - Response time: ~1.5μs

Figure 5.3.3. Table of parts selected for ultrasound detection.

Circuit

The proposed circuit was sketched in LTspice before construction.

Receiver Stage

An AC signal is received by the ultrasonic transducer (400SR10B), which oscillates around 0V with a small amplitude of 270 mV. The transducer output is biased around 2.5V for proper amplification with an opamp with power rails 0V and 5V.

A decoupling capacitor, C1, with capacitance 10nF, is added to pass the 40kHz AC signal while blocking DC offset, so that the amplifier input stays clean at 2.5V. It also acts as a high-pass filter with R2 with a cut-off frequency $\frac{1}{2\pi(10k||10k)*10n} = 3.2kHz$.

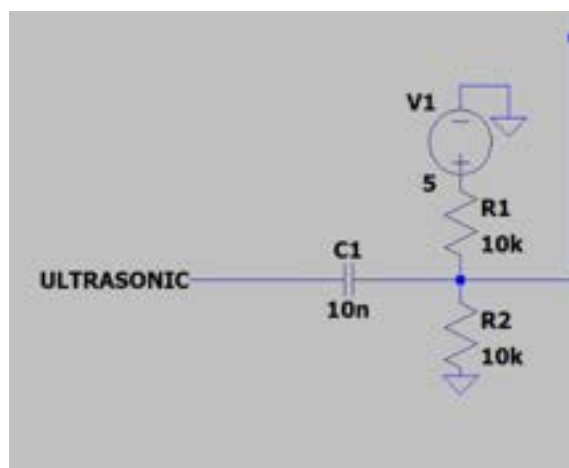


Figure 5.3.4. Ultrasound receiver stage.

Amplification Stage

The opamp was configured as non-inverting with a gain of 111. This was chosen to provide sufficient amplification for the weak signal, while ensuring noise did not falsely trigger a reading. Additionally, the gain was required so that the signal can overcome the forward voltage of the diode.

Filtering was not included in this stage due to the inherent design characteristics of the 400SR10B.

At 40kHz, the impedance of the 400SR10B drops to a minimum and rises steeply on either side of 40kHz. The centre frequency is 40kHz, as desired, and the bandwidth is 3kHz. The ultrasound receiver acts as a bandpass filter by itself, removing the need to build filters.

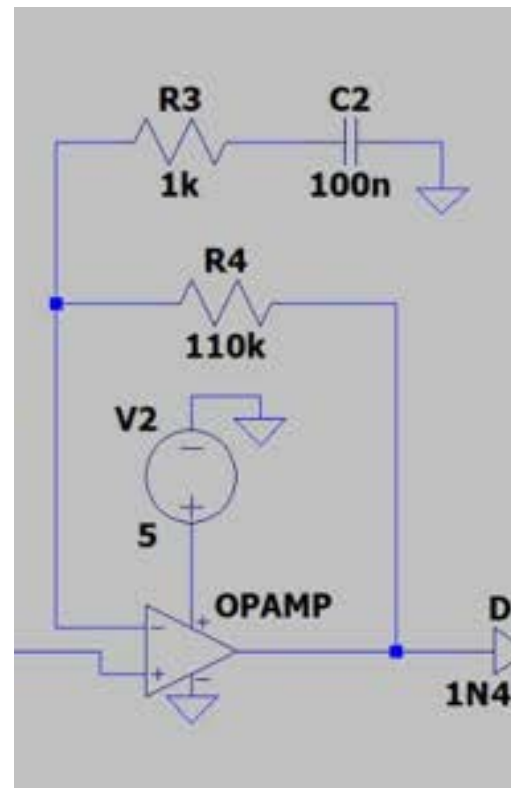


Figure 5.3.5. Amplification stage.

Specification		
400ST100		Transmitter
400SR100		Receiver
Center Frequency		40.0±1.0Khz
Bandwidth (-6dB)	400ST100	2.5Khz
	400SR100	3.0Khz
Transmitting Sound Pressure Level		112dB min.
at 40.0Khz; 0dB re 0.0002μbar per 10Vrms at 30cm		
Receiving Sensitivity		-70dB min.
at 40.0Khz 0dB = 1 volt/μbar		
Capacitance at 1Khz	±20%	1900 pF
Max. Driving Voltage (cont.)		10Vrms
Total Beam Angle	-6dB	72° typical

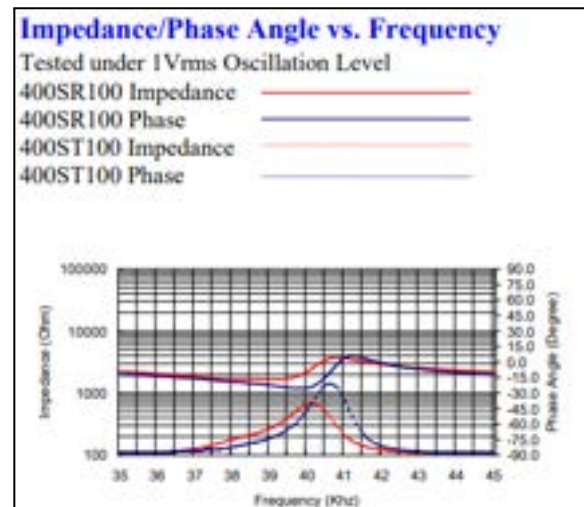


Figure 5.3.6. Datasheet of 400SR10B.

Envelope Detector

The envelope detector comprises D1, R5 and C3. The detector rectifies half of the positive part of the input wave. D1 conducts on the positive half-cycles and charges capacitor C3. R5, connected in parallel, provides the discharge path between peaks. The smoothing time constant is $\tau = R5 * C3 = 10\text{ms}$, which is far longer than the 25μ carrier period, effectively removing the 40kHz ripple.

The detector is DC-coupled to the amplifier output, so the envelope rests on the 2.5 V bias: the no-signal level sits near $(\text{bias} - V_{\text{diode}})$ and rises towards $(\text{signal peak} - V_{\text{diode}})$ when a signal is present. The comparator sets its threshold, 2.3V , within this window.

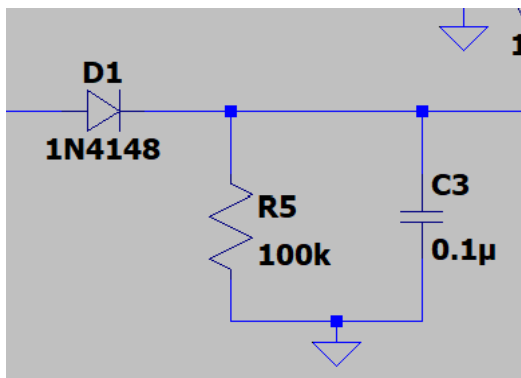


Figure 5.3.7. Envelope detector.

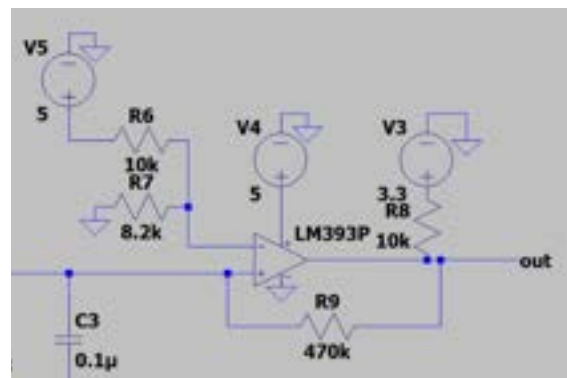


Figure 5.3.8. Comparator stage.

Comparator Stage

The comparator is configured with a reference voltage of 2.3V and a resistor R9 to provide positive feedback. A pull-up resistor R8 connects the comparator output to the 3.3V power supply so that the maximum input into the Metro board is capped at 3.3V , protecting it.

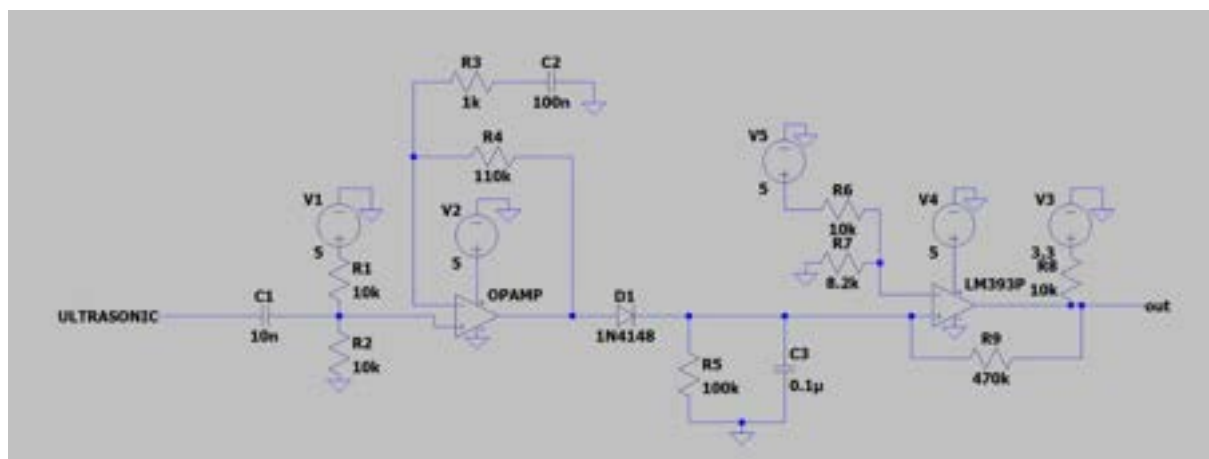


Figure 5.3.9. Overall ultrasonic detection circuit.

Testing

The circuit designed in LTspice was built on a breadboard and tested using an oscilloscope at its various stages.

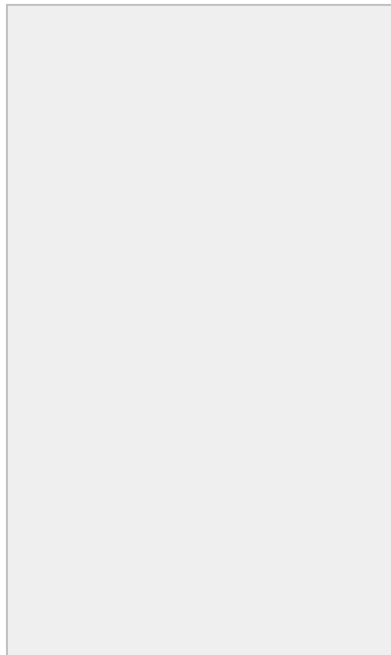
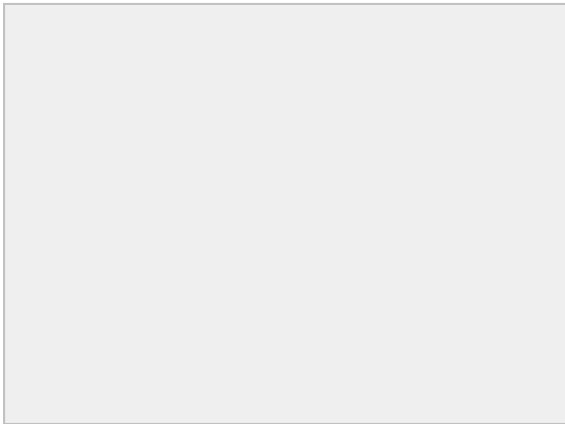
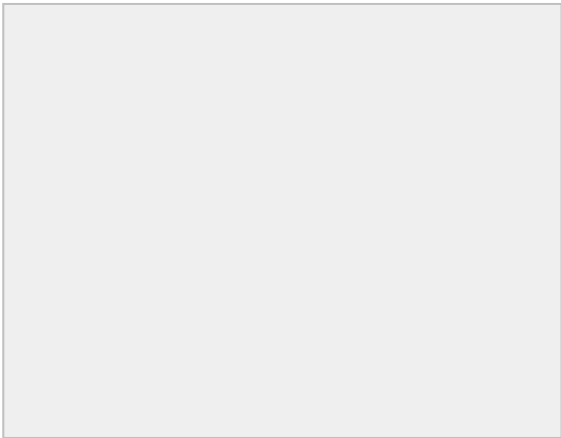
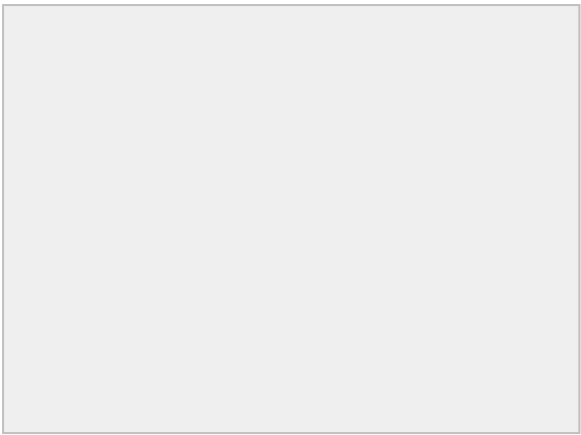
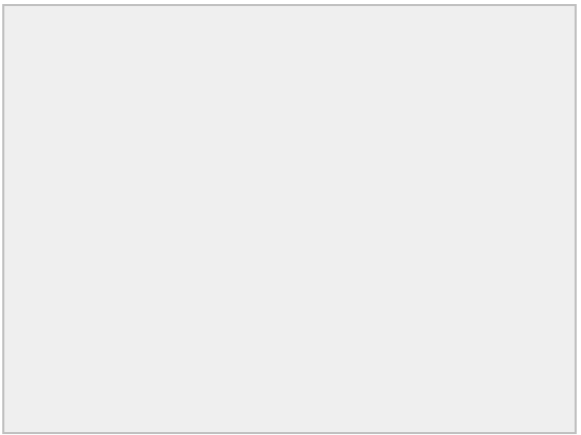


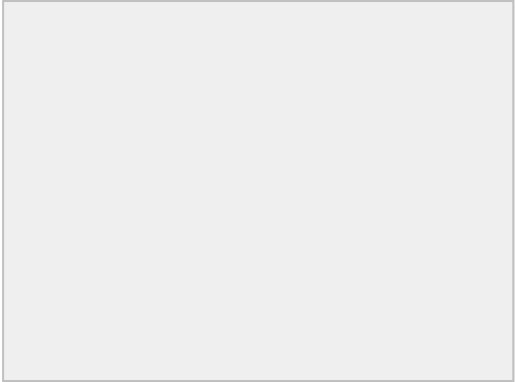
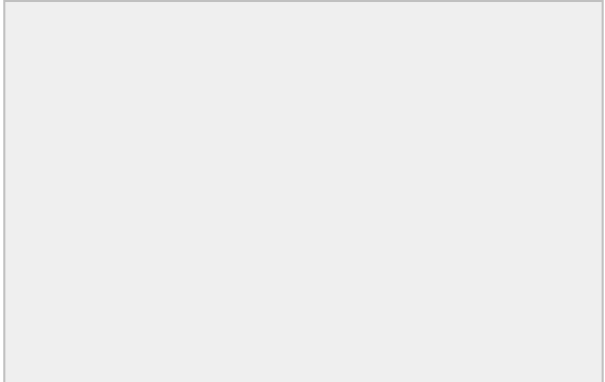
Figure 5.3.10. Ultrasound detection circuit on breadboard.

Before the Amplifier	After the amplifier
 The signal received had a peak-to-peak value of 46.8mV with an amplitude of 23.4 mV, which is not strong enough to pass through the diode.	 After the opamp stage, the peak-to-peak voltage rises to 3.0V. Meaning amplitude is 1.5V, hence allowing the signal to pass through the forward voltage drop of the diode.

Testing at the envelope detector output yielded the following results: When there was no ultrasound signal, the voltage level detected was 2.12V. When the ultrasound signal was present, the voltage level detected was 2.44V.

Absence of rock	Presence of rock
	
Peak voltage level = 2.12V	Peak voltage level = 2.44V

The reference voltage set for the comparator was 2.3V. As intended, a 3.3V output representing a HIGH was detected when the ultrasound signal was present, and a 0V output representing a LOW was detected when the signal was absent.

Before	After
	
Picture of oscilloscope before comparator Its peak value is 200mV, signifying absence of the signal (rock).	Picture of oscilloscope after comparator. Its peak value is 3.8V, signifying the presence of the signal (rock).

Final Implementation

To complete the implementation, the following code was used to output a meaningful user value, and the circuit output was connected to digital pin 2 of the Metro board. A successful output was obtained.

```

UltrasonicRadar.ino
1  void setup() {
2      pinMode(2, INPUT);
3      Serial.begin(9600);
4  }
5
6  void loop() {
7      int ultra = digitalRead(2);
8      if(ultra == 1){
9          Serial.println("Ultrasound detected");
10     }
11     else{
12         Serial.println("No ultrasound detected");
13     }
14     delay(200);
15 }
16

```

Figure 5.3.11. Arduino code used to test ultrasound detector.

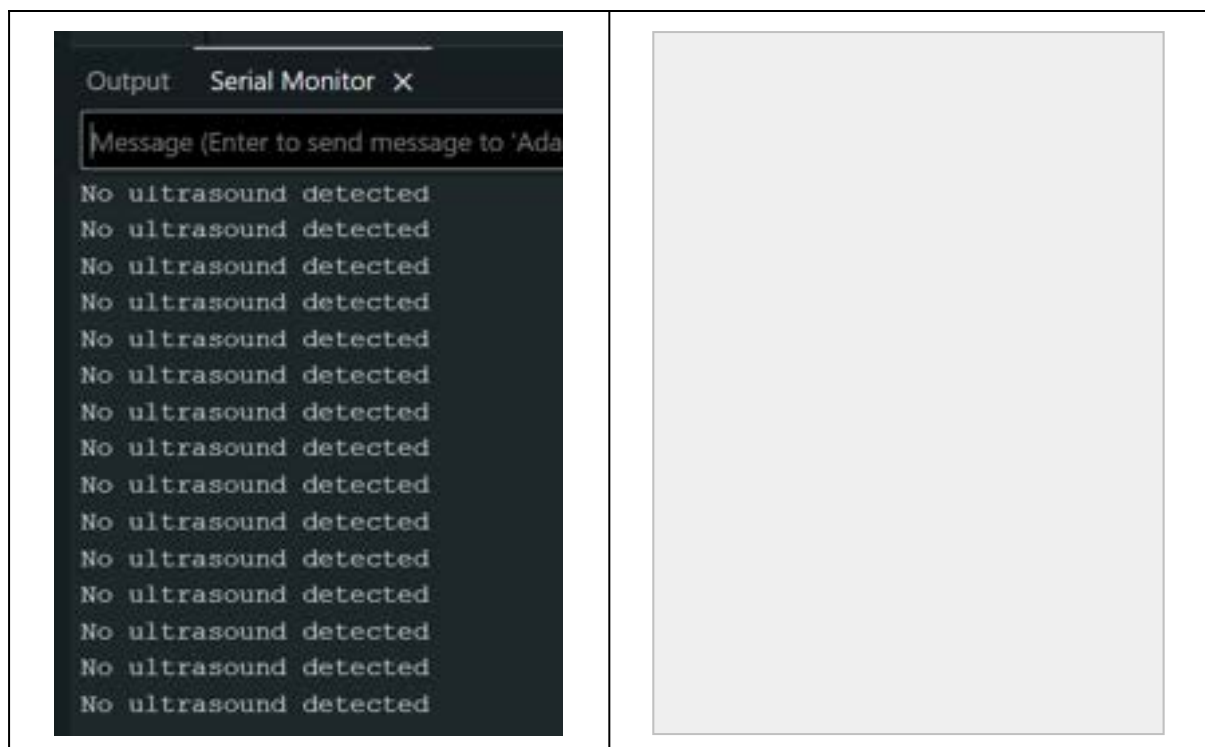


Figure 5.3.12. Serial monitor output with ultrasound signal absence (left) and signal presence (right).

5.4. Radio Detection

Background

The rocks transmit data via a radio signal using 89kHz carrier frequency, which is modulated with two-level amplitude-shift keying (on-off modulation). The age is encoded using ASCII character codes in UART packets with 1 start bit and 1 stop bit. The data rate is 600 bits per second, and each rock's age is four characters long, including an initial # symbol. For example, "#123" means the rock's age is 1.23 billion.

Considerations

The Metro board has built-in UART communication on ports 0 and 1. The Metro board inputs cannot receive voltages greater than 3.3V.

Preliminary Design

The radio signal must first be received by a suitable antenna and then amplified so that the data can be extracted. Envelope detection must be employed to demodulate the signal to produce a signal which indicates the signal's amplitude for proper decoding. A comparator can be used to convert the demodulated signal into a binary signal to be read by the Metro board.

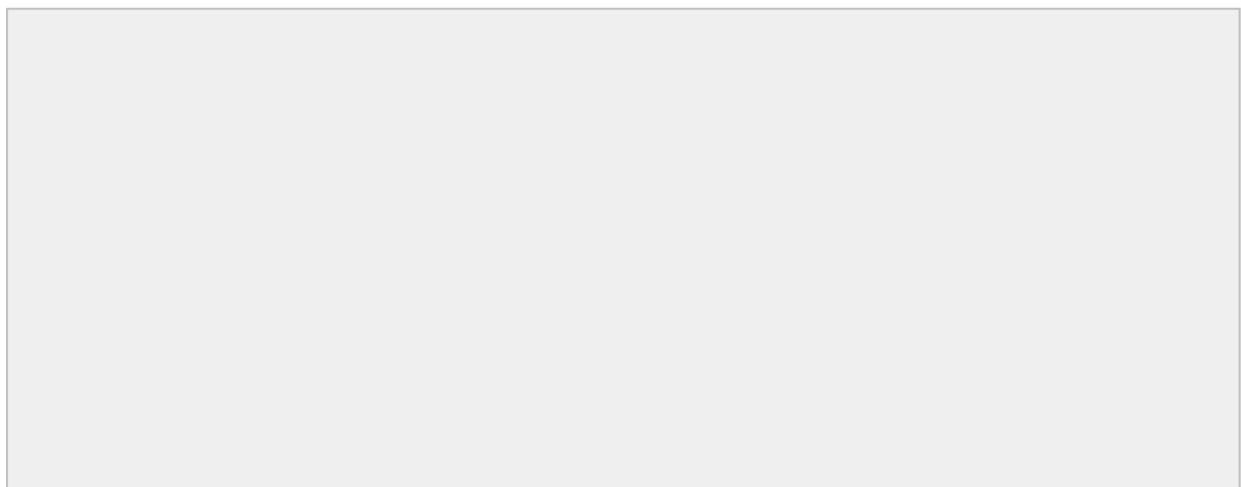


Figure 5.4.1. Radio subsystem detection flow.

A tuned coil antenna, which is essentially an air-cored inductor, can be made by coiling wire to receive the radio signal. The following parts were selected to implement the radio detection circuit.

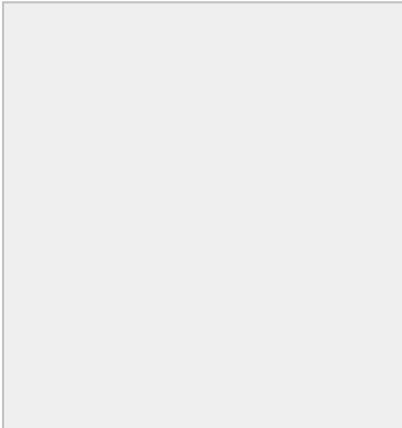
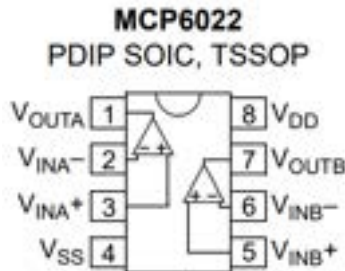
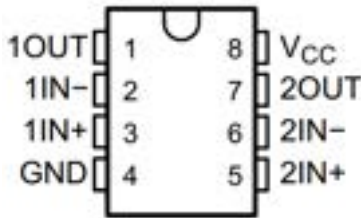
Part	Purpose	Characteristics
Custom wound copper coil 	To receive the radio signal	- Measured inductance of 294.2μH
MCP6022 I/P 	Opamp to amplify the output from the antenna	- Power supply range: 2.5-5.5V - Gain-bandwidth product: 10MHz (since the target frequency is 89kHz, a maximum gain of 10M/89k = 112 is possible) - Slew rate: 7V/μs - Dual channel
1N4148 	Diode used for the envelope detector	- Fast-switching; 5ns reverse Recovery - Forward voltage max: 1V - Operating Temperature Max: 175 °C
LM393P 	Comparator to convert the output from the MCP6022 into a clean digital signal that the Metro can read	- Open-drain NPN transistor output - Single supply compatible: 2-36V - Response time: ~1.5μs

Figure 5.4.2. Table of parts selected for radio detection.

Circuit

The proposed circuit was sketched in LTspice before construction.

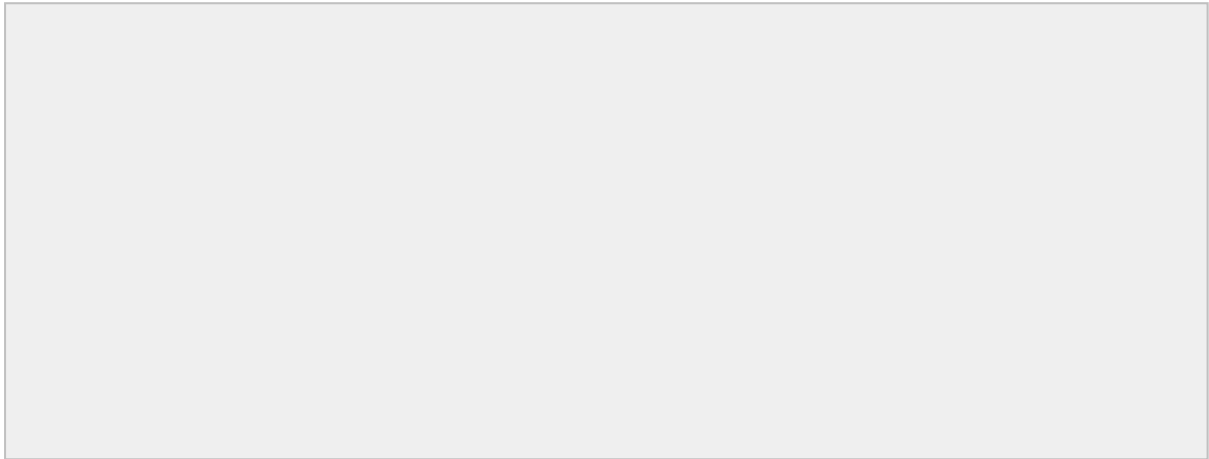


Figure 5.4.3. LTspice radio detection circuit.

The radio antenna is modelled using two nodes, with one connected to ground and the other as the input to the circuit.

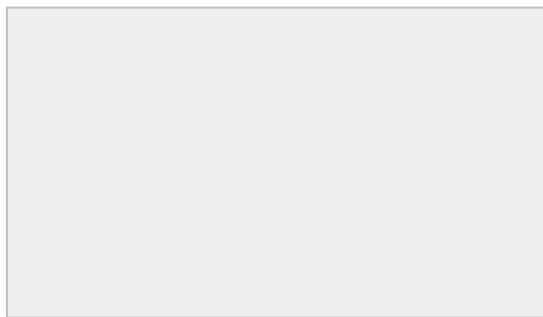


Figure 5.4.4. Radio antenna stage.

The radio antenna was hand-wound using copper enamelled wire. It was measured to have an inductance of 294.2μH using an LCR meter. A capacitor of 10nF was connected in series to the antenna input to tune the antenna to the resonant frequency of 89kHz. This value was selected using the resonance formula:

$$f = \frac{1}{4\pi^2\sqrt{LC}} \rightarrow C = \frac{1}{4\pi^2Lf^2} = \frac{1}{4\pi^2(294.2*10^{-6})(89000)^2} = 10.9nF \approx 10nF.$$

The amplification stage was configured as shown below. A non-inverting amplifier was set up with a gain of 63. Before amplification, the input signal is biased around 2.5V so that both negative and positive sides of the input signal can be amplified with the opamp power rails of 0V and 5V.

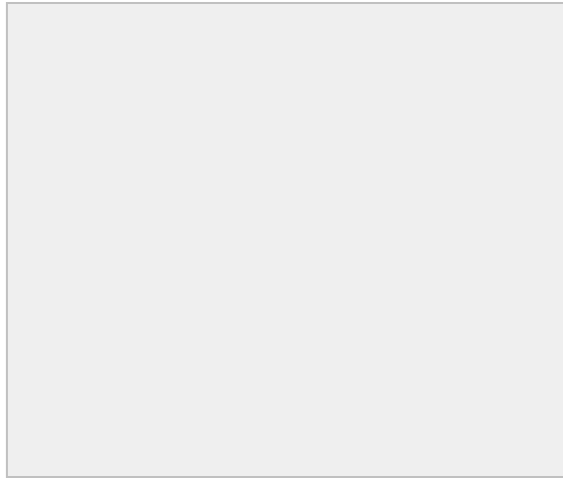


Figure 5.4.5. Amplification and biasing stage.

The envelope detector was designed using D1, R4 and C3. The 1N4148 diode rectifies the positive half-cycles and charges a capacitor, which discharges through a parallel resistor between peaks. $R4 = 10k\Omega$ and $C3 = 10nF$ were chosen to give a time constant of $\tau = RC = 10k * 10n = 100\mu s$. This time constant is longer than the $11\mu s$ carrier period so that the 89kHz ripple is smoothed away but shorter than the 1.67mb bit period so that the edges of the data waveform are preserved.

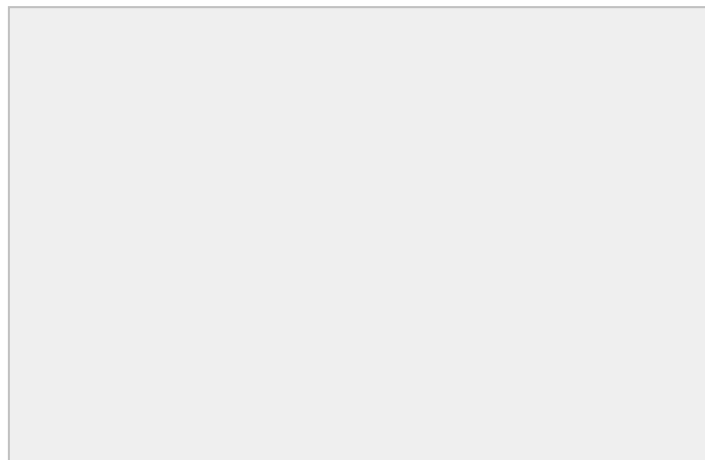


Figure 5.4.6. Envelope detector stage.

The final comparator stage was configured with a reference voltage of 2.3V and a feedback resistor R5 to provide positive feedback, to clean up comparator chatter and ensure no false switching. A pull-up resistor R7 connects the comparator output to the 3.3V power supply so that the maximum input into the Metro board is capped at 3.3V, protecting it.

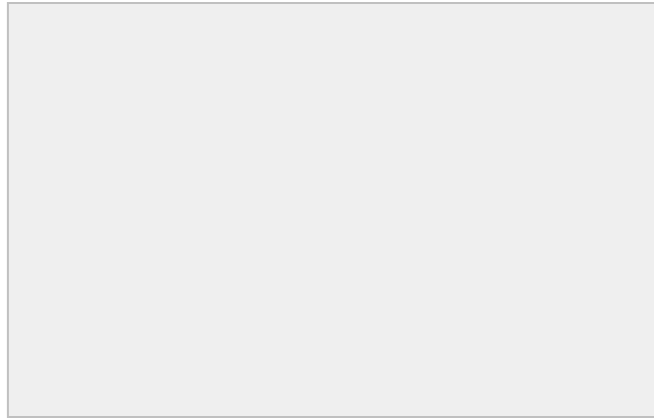


Figure 5.4.7. Comparator stage.

Testing

The circuit was built on a breadboard, and an oscilloscope was used together with the rock simulator for testing at the various stages.

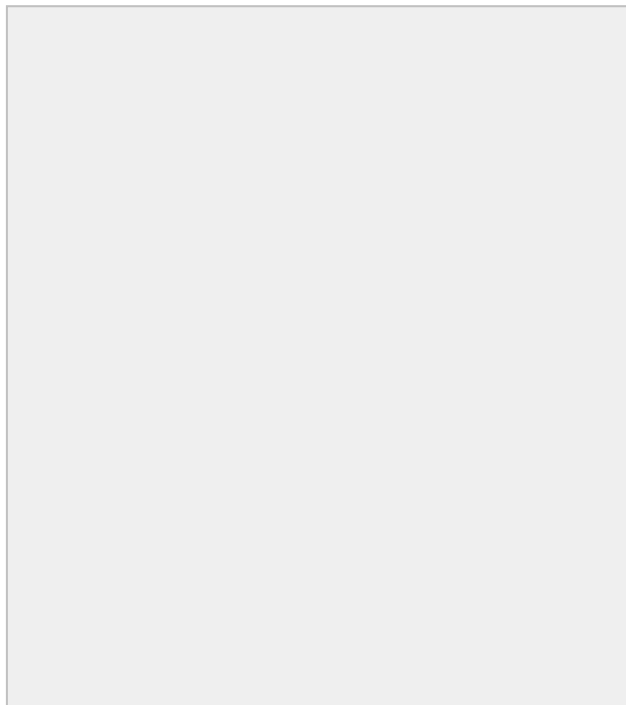


Figure 5.4.8. Radio detector circuit on breadboard.

The peak-to-peak value of the input signal was measured as 80 mV, which was successfully amplified to 1.92V. The gain was 24, which is sufficient to overcome the forward voltage of the diode but less than the predicted gain of 63. This could be due to the breadboard circuitry. The signal was also correctly biased around 2.5V.

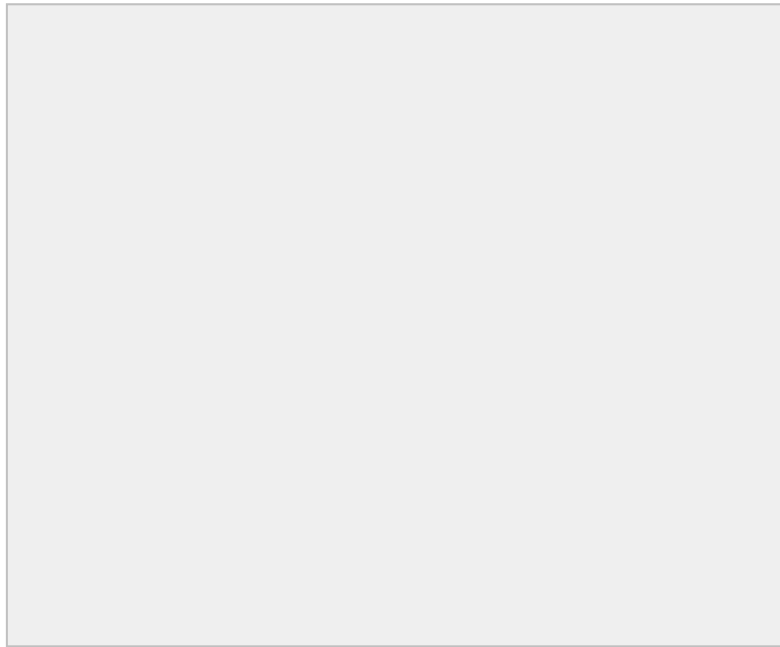


Figure 5.4.9. Input and amplified radio signal.

The envelope detector worked as intended, smoothing the edges and obtaining the outline of the radio signal, shown correctly as pulses. The comparator output produced a clean output with a maximum value of 3.3V for a correct binary signal input into the UART port of the Metro board.

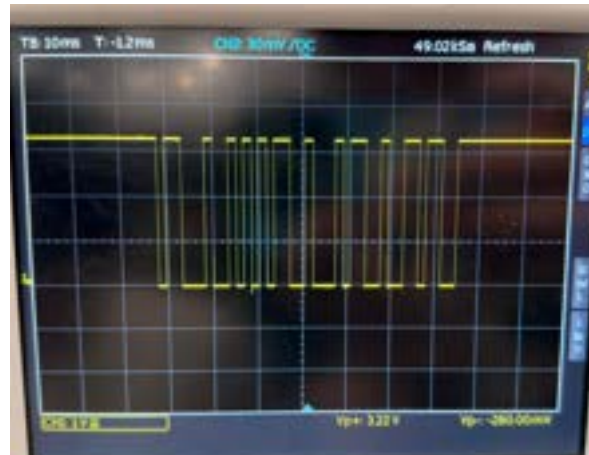
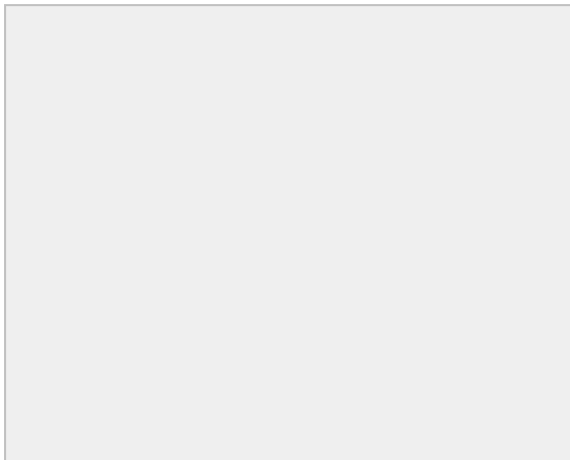


Figure 5.4.10. Envelope detector (left) and comparator output (right).

Final Implementation

To complete the implementation, the radio detection circuit was wired to pin 0, the Metro board UART port and the following code was written. The code sets the UART port bit rate to 600, in line with the data rate of the radio signal. When data is received by the UART port, it is read and interpreted. When a “#” is received, this indicates the start of a new age. The following characters are written out as the format of the age is a “#” followed by three digits.

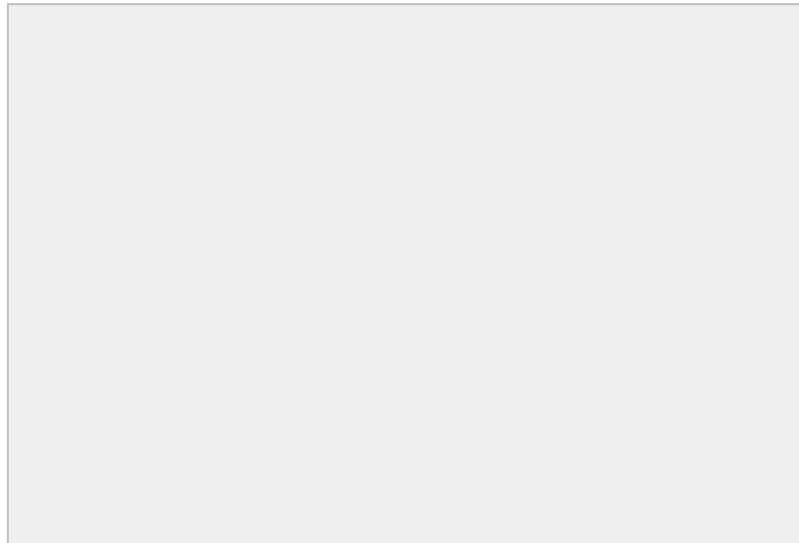


Figure 5.4.11. Arduino code for the radio detection test.



Figure 5.4.12. Serial monitor output for age detection test.

Testing with three different rock age simulations showed the radio detector working as intended.

6. Robot Design

This section of the report covers in-depth explanations of the design choices, testing and final implementation of the rover movement and software subsystems:

6.1. Rover Chassis

6.2. Software

6.1. Robot Chassis

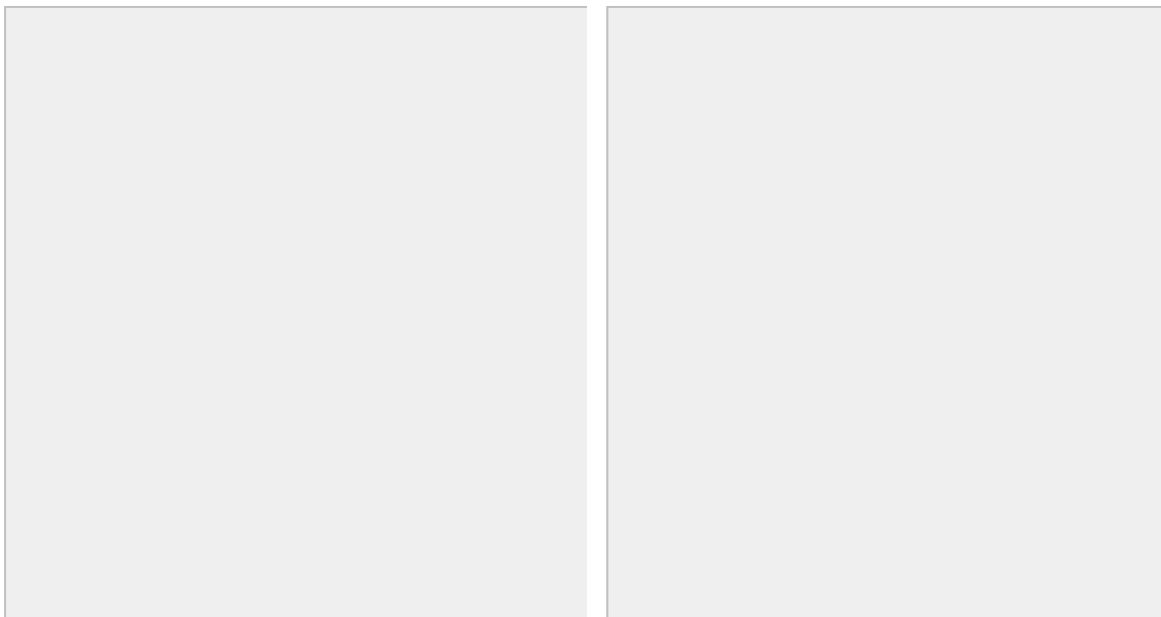


Figure 6.1. EEEBug base chassis.

Figure 6.2. Top-down view of EEEBug.

The EEEBug base chassis was used to implement the robot design. The Metro board and wifi shield were mounted at the back of the EEEBug acrylic chassis, and the motor driver module was plugged into the EEEBug PCB. The drive motors were accordingly wired to the motor driver module. The EEEBug batteries provided the power needed for the Metro board, the motors and the sensor subsystems.

At the front of the EEEBug chassis, a breadboard was mounted, enabling sensor subsystems to be added onto the breadboard later.

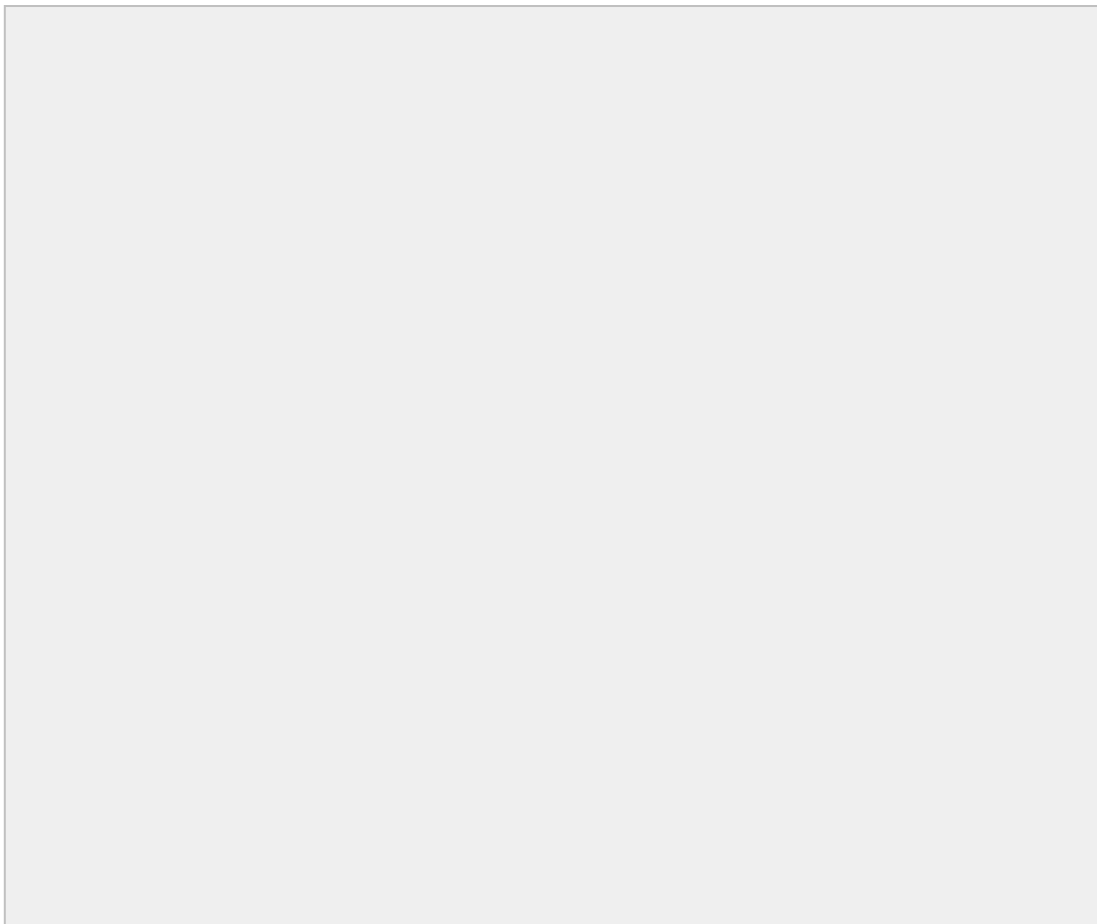
6.2. Software

The rover's software runs on the Metro M0 Express and is in charge of 3 key roles:

1. Hosting a web-based control interface over a WiFi network.
2. Controlling the motors based on the controls received from the stage above.
3. Reading and analysing the data gathered from sensors to classify the rock type and display the result back to the user.

WiFi communication and web server

The control interface will be hosted from the M0 board, and is accessible via the selected network. The rover can be controlled from any device on the same network by entering the appropriate IP address on an internet browser. Initially, this was set to the shared EEERover network, but was later configured to use a hotspot connection to improve latency due to high traffic on the EEERover network.

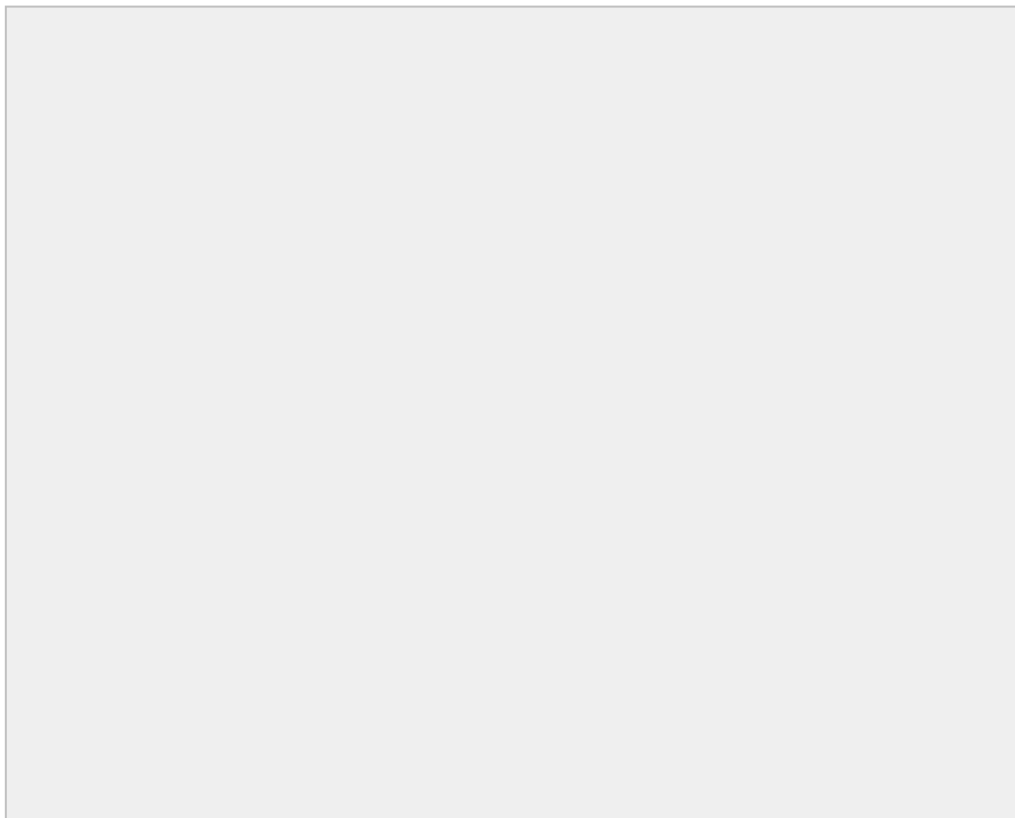


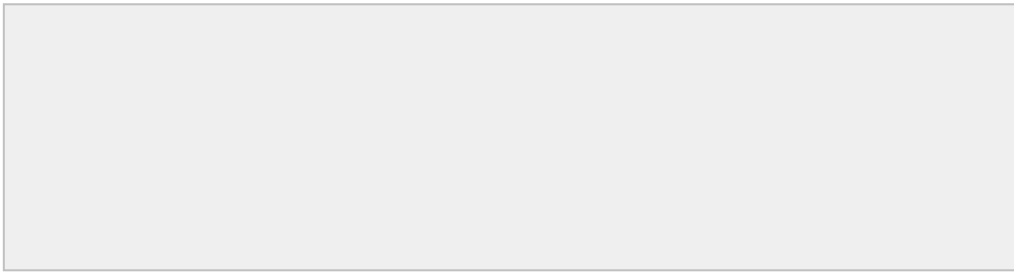
For the network, two options were considered: EE Lab's EEERover Network or a mobile phone hotspot network. To compare the two options, the latency between the time of control on a device and the time of control data received by the rover was investigated. While the EEERover network showed a 220ms round-trip time, the mobile phone's hotspot network showed 40~80ms latency. Consequently, a mobile phone hotspot was chosen as the network hosting. The implementation of this external connectivity solution remained compliant with project guidelines, as the initial documentation provided explicit permission for such alternative network configurations.

To prevent the IP address from constantly changing between every session, the command `WiFi.config` was used to assign a fixed address of 172.20.10.10. This allows the user to type in the same IP address for the control interface every time they wish to access it. During the setup stage for the M0 board, `server.on()` registers a handler for each command and data URI, and each handler returns a short acknowledgement back to confirm that the command has been executed. Then `server.begin()` command line starts the server and to service continuously incoming requests, `server.handleClient()` is called on every iteration of the `loop()`.

Movement Control

Motor control is built around a single `setMotor()` routine. The sign of its routine's parameter sets the direction of each wheel (forward or reverse), and the speed of the wheel is represented by the amplitude, which is selected using the slider shown on the web interface.





Driving commands map onto wheel combinations, a forward command would drive both wheels at the same time in the same sign&magnitude. A turn command would drive the two wheels in opposite directions so that the rover rotates on the spot rather than circling around.

A communication failsafe exists so that the rover is stopped if no command is received for more than 500ms. This prevents losing control of the rover when the WiFi connection becomes unstable.

Memory and page-delivery optimisation

Upon researching the board's specification, it was discovered that the M0 Express board only has 32KB of RAM, which would not be large enough to hold all the HTML data required to render the control interface. Therefore, information was stored in flash memory using `PROGMEM`, and short fixed strings were wrapped in F() to keep them out of RAM memory.

```
5  const char webpage[] PROGMEM = R"rawliteral(  
6  <!DOCTYPE html>  
7  <html>  
8  <head> ... CSS styles ... </head>  
9  <body>  
10 |   <!-- direction buttons, the speed slider, the live sensor dashboard, and the  
11 |   JavaScript that sends drive commands and polls the sensors -->  
12 </body>  
13 </html>  
14 )rawliteral";  
15  
16 // Short, fixed strings use the F() macro for the same reason:  
17 // F(...) is also kept in flash instead of being copied into RAM.  
18 void replyAPI(const String& msg) {  
19   server.send(200, F("text/plain"), msg); // the "text/plain" literal stays in flash  
20 }
```

```

5 void handleRoot() {
6     server.setContentLength(CONTENT_LENGTH_UNKNOWN);
7     server.send(200, F("text/html"), ""); // send the header first; the body follows in chunks
8     int totalLength = sizeof(webpage) - 1;
9     int bytesSent = 0;
10    const int CHUNK_SIZE = 512;
11    char chunkBuffer[CHUNK_SIZE + 1];
12    while (bytesSent < totalLength) {
13        int n = (totalLength - bytesSent > CHUNK_SIZE) ? CHUNK_SIZE : totalLength - bytesSent;
14        memcpy_P(chunkBuffer, webpage + bytesSent, n); // copy one chunk OUT of flash (PROGMEM) into RAM
15        chunkBuffer[n] = '\0';
16        server.sendContent(chunkBuffer); // send that 512-byte chunk
17        bytesSent += n;
18        delay(20); // let the WiFi buffer catch up
19    }
20    server.sendContent(""); // empty chunk = end of the response
21 }

```

Upon writing the code, a second constraint, the throughput of the board, was encountered. The board's specification states that the board transmits roughly 1KB per second, however the full HTML page is 8KB in size. Sending this data in a single response would stall the connection and prevent the page from being sent. A solution around this was to split the page into smaller chunks of sizes at most 512 bytes, sent sequentially with a short delay between each packet. This allowed the page to load successfully, taking roughly 0.5s.

Sensor Integration and Data Fusion

Inside the main loop, each subsystem is executed, and the results are combined to classify the type of rock. The data processing structure for each subsystem was designed according to its operational characteristics.

```

1 // IR is a stream of FAST pulses, so we use a hardware INTERRUPT to never miss one.
2 const int IR_PIN = 9;
3 volatile unsigned long pulseCnt = 0; // 'volatile': it is changed inside the interrupt
4 unsigned long lastIRTime = 0;
5 float IRpulseRate = 0;
6
7 void countPulse() { pulseCnt++; } // the ISR: just +1 per pulse (kept tiny on purpose)
8
9 void setup() {
10 // every rising pulse on IR_PIN automatically calls countPulse()
11 attachInterrupt(digitalPinToInterrupt(IR_PIN), countPulse, RISING);
12 }
13
14 void loop() {
15 // Every ~500ms, turn the pulse COUNT into a RATE (pulses per second)
16 unsigned long elapsed = millis() - lastIRTime;
17 if (elapsed >= 500) {
18 noInterrupts(); // pause interrupts to read the shared counter safely
19 unsigned long cnt = pulseCnt; pulseCnt = 0;
20 interrupts();
21 lastIRTime = millis();
22 IRpulseRate = (float)cnt * 1000.0 / (float)elapsed; // rate = count / real elapsed time
23 }
24 }
25
26 // The rate maps to a tier that the rock-type check uses:
27 String irStatus() {
28 if (IRpulseRate > 400) return "547";
29 else if (IRpulseRate >= 240) return "312";
30 else return "No IR";
31 }

```

Since infrared pulses are short and frequent, polling data would risk sampling during a gap and missing counts. Instead, an interrupt on the rising edges of the pulse signal was implemented to guarantee that every pulse is registered.

```

1 // Magnetism: a Hall sensor on an analogue pin. We just threshold the reading.
2 const int MAG_PIN = A4;
3 bool magIsNorth(int v) { return v > 620; }
4 bool magIsSouth(int v) { return v < 600; } // 600-620 is a dead zone -> "None"
5
6 String magStatus() {
7 int v = analogRead(MAG_PIN);
8 if (magIsNorth(v)) return "UP (North)";
9 if (magIsSouth(v)) return "DOWN (South)";
10 return "None";
11 }
12
13 // Ultrasound: a digital pin. HIGH = 40kHz detected, LOW = nothing.
14 const int US_PIN = 12;
15 String usStatus() { return digitalRead(US_PIN) == HIGH ? "Detected" : "None"; }

```

Magnetic and ultrasonic data sampling are easier than others, since they only require basic condition check statements. The magnetic channel thresholds an analogue reading against the deadband around its baseline ; a value above the band indicates an upward field, whereas a value below the band would indicate a downward field, finally a value within the band indicates that there are no fields present. An ultrasonic channel can be read by a single channel read where a HIGH logic signal would indicate the presence of an ultrasonic signal.

```

1 // Radio sends the age as text over a UART (Serial1), e.g. "#444#444#...". The line is
2 // noisy, so we never trust one reading; we keep every clean 3-digit reading from the
3 // last 2 seconds and show the MODE (most frequent). Noise (44 / 445 / 4443) loses the vote.
4 // ageVals[] + ageTimes[] = a 64-slot ring buffer of recent readings and their arrival times.
5
6 // In loop(): read the UART; any non-digit ends a reading; store only 3-digit ones.
7 while (Serial1.available() && ageBudget-- > 0) {
8   char c = Serial1.read();
9   if (c >= '0' && c <= '9') { if (ageLen < AGE_BUF - 1) ageBuf[ageLen++] = c; }
10  else { // non-digit ('#', noise) = end of a reading
11    if (ageLen == 3) { // keep only 3-digit readings
12      ageVals[ageHead] = (ageBuf[0]-'0')*100 + (ageBuf[1]-'0')*10 + (ageBuf[2]-'0');
13      ageTimes[ageHead] = millis();
14      ageHead = (ageHead + 1) % AGE_N;
15    }
16    ageLen = 0;
17  }
18 }
19
20 // The displayed value = the most frequent reading in the last 2 seconds (the mode):
21 String ageBest() {
22   int bestVal = -1, bestCount = 0;
23   for (int i = 0; i < AGE_N; i++) {
24     if (ageTimes[i] == 0 || millis() - ageTimes[i] > AGE_WIN) continue; // skip empty/old
25     int c = 0;
26     for (int j = 0; j < AGE_N; j++)
27       if (millis() - ageTimes[j] <= AGE_WIN && ageVals[j] == ageVals[i]) c++; // count matches
28     if (c > bestCount) { bestCount = c; bestVal = ageVals[i]; }
29   }
30   return bestVal < 0 ? "no signal" : withDecimal(bestVal); // 444 -> "4.44"
31 }

```

Radio decoding required the most work to build, the challenge was on how to stably display the age without any errors. The initial code could only print an output that had consecutive numbers in it, and sometimes even lost some letters during reading.

To solve this, two conditions were added, when a `#` is detected, it would refresh the displayed value and it would only frame when three digits are accepted. The 3-digit filter removes corrupted and two-digit readings; among the valid 3-digit readings, a voting scheme over 2 seconds picks the most frequent value, so single-digit errors like 441/443/445 are outvoted. This allows the display to look stable at the cost of a 2-second settling period when the rock is changed.


```

1  // — Rover side: pack EVERY sensor reading into ONE response, separated by '|' —
2  // One request instead of five -> much less traffic competing with the drive commands.
3  void handleSensors() {
4      String out = irStatus() + " - " + String((int)IRpulseRate) + "/s";
5      out += "|" + magStatus();
6      out += "|" + usStatus();
7      out += "|" + ageBest();           // age, e.g. "4.44"
8      out += "|" + rockType();         // rock type
9      replyAPI(out);                   // e.g. "312 - 298/s|None|Detected|4.44|Gravion"
10 }
11
12 // — Browser side: poll that one endpoint, split it, fill in the dashboard —
13 setInterval(function () {
14     var sx = new XMLHttpRequest();
15     sx.open('GET', '/sensors', true);
16     sx.onload = function () {
17         var p = sx.responseText.split('|');           // [ir, mag, us, age, rock]
18         document.getElementById('ir-val').innerHTML = p[0];
19         document.getElementById('mag-val').innerHTML = p[1];
20         document.getElementById('us-val').innerHTML = p[2];
21         document.getElementById('age-val').innerHTML = p[3];
22         document.getElementById('rock-id').innerHTML = p[4];
23     };
24     sx.send();
25 }, 250);                                           // every 250ms

```

Rock classification combines the three types of sensor readings that help identify the type of rock: infrared rate, ultrasonic presence and magnetic polarity. Each rock type corresponds to a unique combination of the three, and any two of the three sensor readings are sufficient to correctly identify the rock. Hence, rather than requiring an exact match on all three signals, the firmware counts how many of each type's expected features are present and selects the type that has at least two matches. This allows identification of the rock even when one sensor is faulty.

```

1  // — Rover side: pack EVERY sensor reading into ONE response, separated by '|' —
2  // One request instead of five -> much less traffic competing with the drive commands.
3  void handleSensors() {
4      String out = irStatus() + " - " + String((int)IRpulseRate) + "/s";
5      out += "|" + magStatus();
6      out += "|" + usStatus();
7      out += "|" + ageBest();           // age, e.g. "4.44"
8      out += "|" + rockType();         // rock type
9      replyAPI(out);                   // e.g. "312 - 298/s|None|Detected|4.44|Gravion"
10 }
11
12 // — Browser side: poll that one endpoint, split it, fill in the dashboard —
13 setInterval(function () {
14     var sx = new XMLHttpRequest();
15     sx.open('GET', '/sensors', true);
16     sx.onload = function () {
17         var p = sx.responseText.split('|');           // [ir, mag, us, age, rock]
18         document.getElementById('ir-val').innerHTML = p[0];
19         document.getElementById('mag-val').innerHTML = p[1];
20         document.getElementById('us-val').innerHTML = p[2];
21         document.getElementById('age-val').innerHTML = p[3];
22         document.getElementById('rock-id').innerHTML = p[4];
23     };
24     sx.send();
25 }, 250);                                           // every 250ms

```

Finally, reporting was optimised for responsiveness. Initially, each value was sent in its own packet once per second, producing 5 packets per second and a 1-second update latency. The values are now combined into a single packet transmitted every 0.25s, reducing the packet rate from 5 to 4 per second while improving the update latency from 1s to 0.25s.

7. Complete Rover

Integration

Following successful verification of each sensor subsystem's functionality, the individual sensor subsystems were transferred onto a shared breadboard. To enable a seamless transfer of the four detection circuits, the circuit was first designed using Fritzing, as shown below.

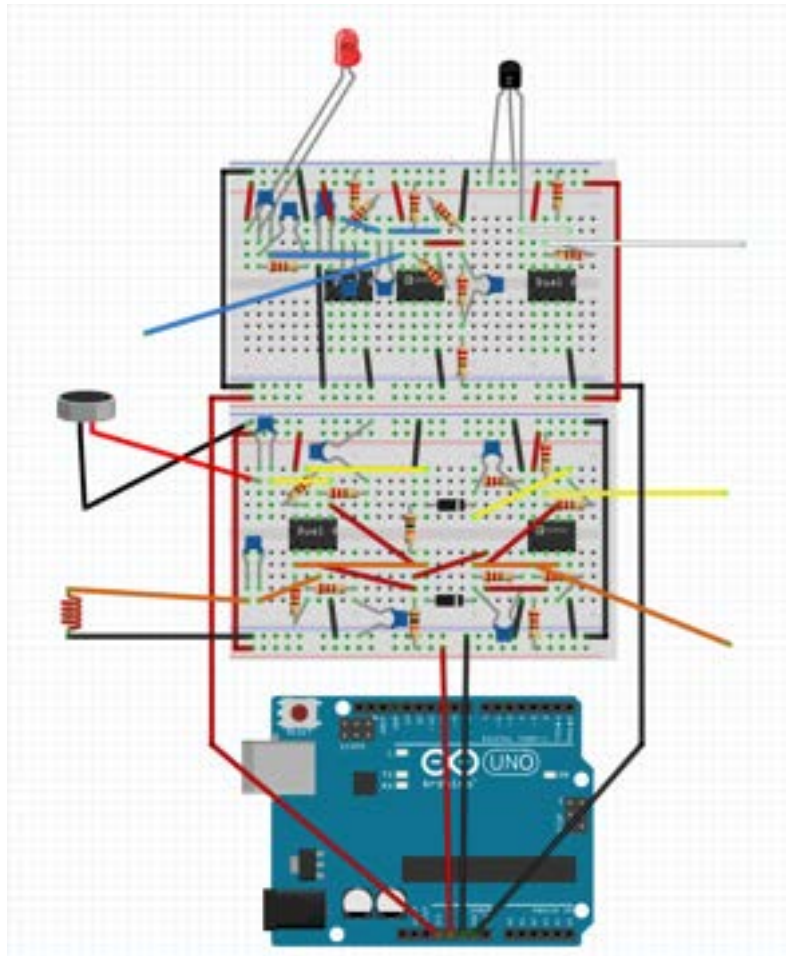


Figure 7.1. Fritzing circuit of combined integrated sensor subsystems.

The final circuit was then built on two combined breadboards. The radio antenna, 400SR10B ultrasound transducer, TEFT4300 infrared phototransistor and DRV5053RAQLPG hall effect sensor were soldered to wires to enable for mounting from a further distance from the central breadboard. The sensors and circuit breadboard were then mounted on the rover chassis as shown.

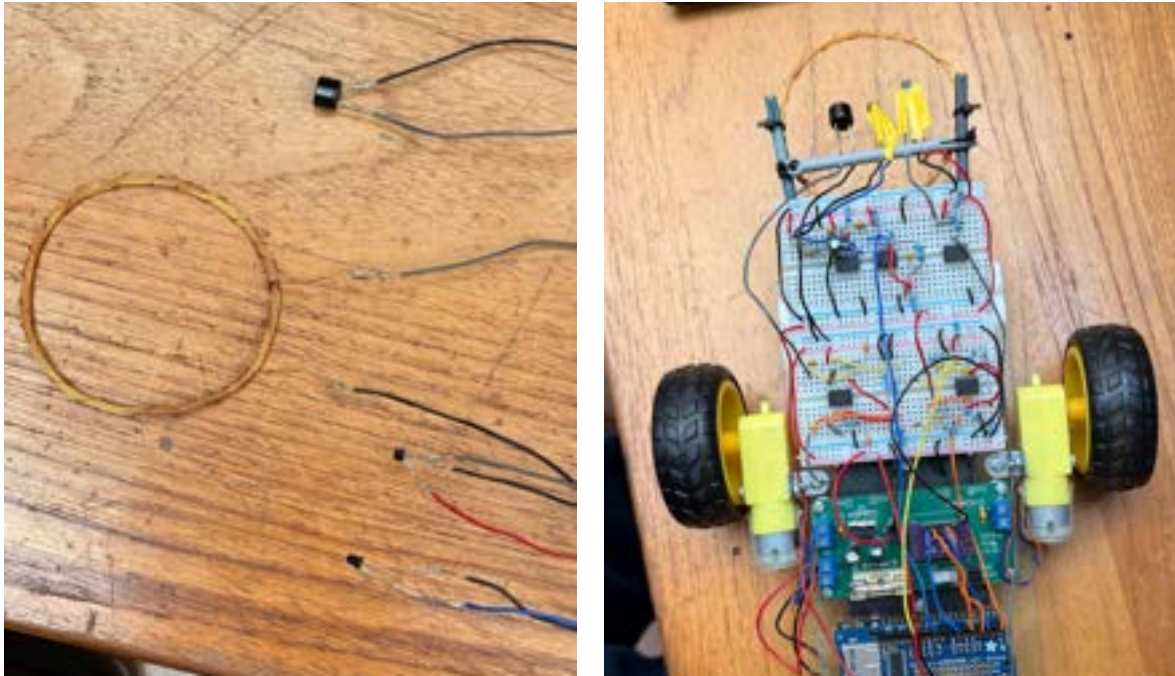


Figure 7.2. Soldered sensors (left) and fully mounted breadboard and sensors (right).

3-D printed prongs were used to mount the radio antenna at the front of the rover, as far away from the Metro board to prevent any possible interference from the wifi communication from the Metro and the WiFi shield. The remaining sensors were similarly mounted using an arch-bracket system so that collectively, they would be in a prime position to detect all the signals when the rock is positioned centrally underneath the sensor mount area.

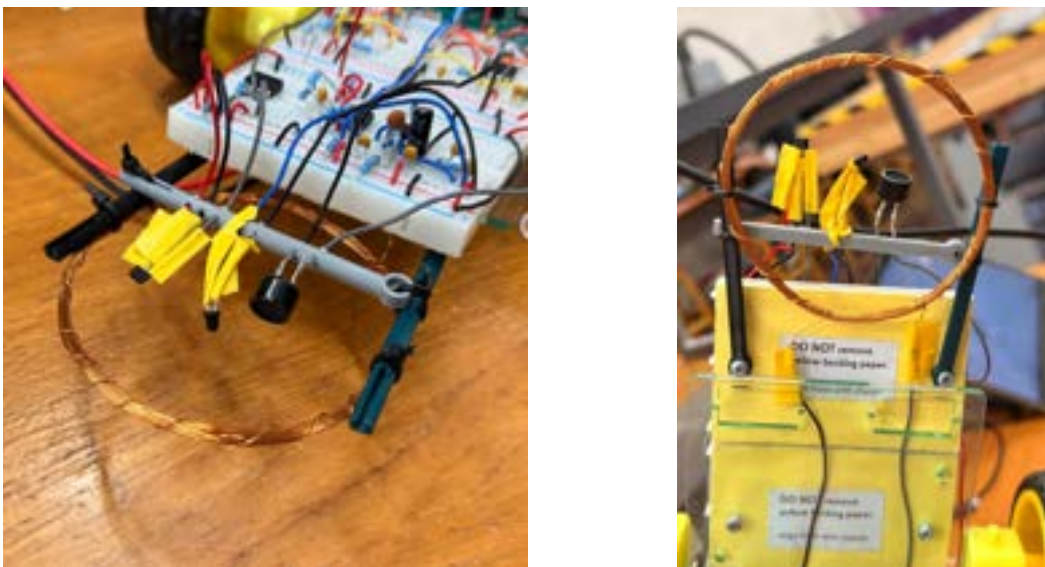


Figure 7.3. Close-up of sensor mount.

Zip-ties were used to mount the antenna to the prongs as they are lightweight and non-conductive and would not alter the tuned inductance of the antenna. Holes were drilled into the prongs to enable the wires of the sensors to pass through and be organised clearly. Yellow electrical tape was used to prevent short circuit connections and to add extra construction robustness to prevent the soldered wires from accidentally breaking.

Testing

Each detection subsystem was first verified independently against the rock simulator.

Subsystem	Test Method	Key Measured Result	Outcome
Magnetic	Test Magnet, both polarities	Baseline ≈ 1.94 V Down $\Rightarrow 0.4$ V Up $\Rightarrow 3.3$ V	Magnet direction found within 2~3cm.
Infrared	Measure the IR rate for different class of rocks	Distinguishes between two rates successfully.	Two rates correctly distinguished
Ultrasound	Measure the presence of 40kHz ultrasound signal	Envelope 2.12V (absent) vs 2.44V (present); clean comparator output at 2.3V threshold	Presence/Absence reliably detected
Radio	Measure the value being emitted from Radio signal	Coil $\approx 48\text{--}60$ mV $\rightarrow$ amplified to 928mV $\rightarrow$ UART decode	Age decoded correctly for three rocks

The core requirement of the rover is to classify all four rock types. No single signal achieves this, but the three independent measurement subsystems form a unique combination for each type of rock, as shown below.

<u>Type</u>	<u>Infrared</u>	<u>Ultrasound</u>	<u>Magnetic</u>
Basaltoid	547s^{-1}	Present	Down
Gravion	312s^{-1}	Absent	Down
Regolix	312s^{-1}	Present	Up
Lunarite	547s^{-1}	Absent	Up

The rover was then tested on the demo surface. Using the web page user control interface, the rover was capable of navigating the obstacles on the map, a video of which can be found in the project Github repository. Testing of the classification ability was conducted using the provided rock simulator and adjusting the direction of the included magnet within the rock casing.

1. Basaltoid

1234

■ ■ ■ ■

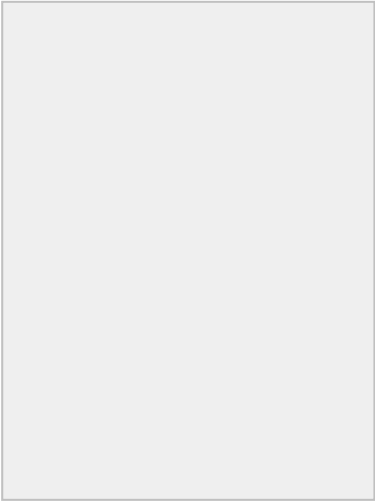
$f=40\text{kHz}$

Age of Rock

$\lambda = 547\text{ s}^{-1}$

Downward magnetic field





Lunar Rover PRO V5

Connected • 48 ms (min. 37)

4.48 billion years old Basaltoid

⬅

⬆

➡

⬅

STOP

➡

⬆

⬇

⬆

➡

Speed: 0.00 / 255

Sensor Dashboard

Radio

4.48 billion years

Magneto

0.000 (0.001) = 382

Infrared

0.01 = 400.0

Ultrasonic

Detected

51

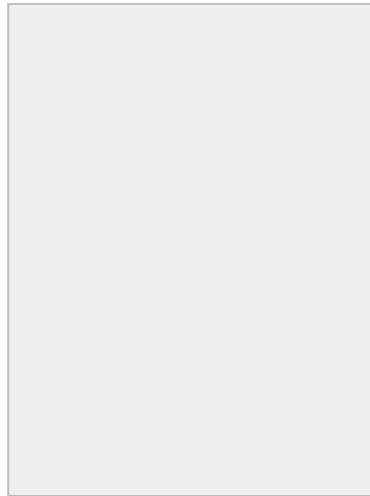
2. Gravion



Age of Rock

$$\lambda = 312\text{s}^{-1}$$

Downward magnetic field



3. Regolix

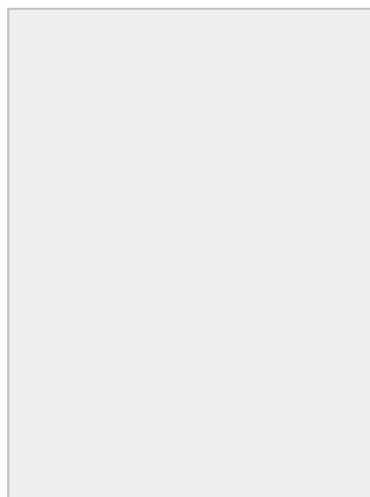


$f=40\text{kHz}$

Age of Rock

$$\lambda = 312\text{s}^{-1}$$

Upward magnetic field



4. Lunarite

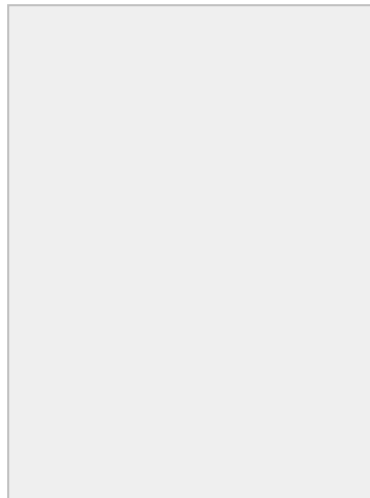


$f=40\text{kHz}$

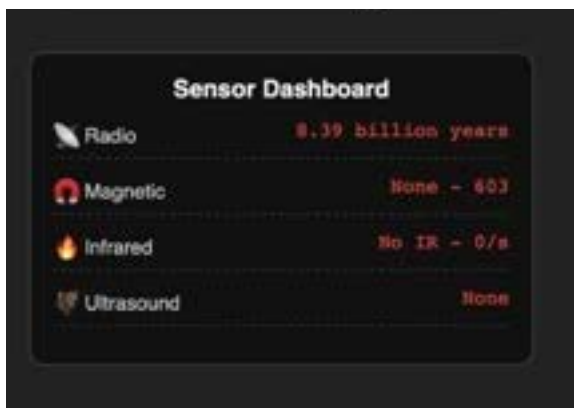
Age of Rock

$\lambda = 547\text{s}^{-1}$

Upward magnetic field



All rock types were successfully classified. Additional testing was performed to verify that random ages could be detected, as shown below:



Besides the functionality of the rover, the following non-functional requirements were also achieved, proof of which can be seen in the supplementing evidences table.

<u>Requirement</u>	<u>Test</u>	<u>Result</u>
Weight Restriction; Must be lighter than 750g	Final assembled rover weighed on a scale	494.5g [SE1]
Manoeuvrability	Rover driven through an obstacle course	Able to navigate around small gaps [SE2]
	Rover able to rotate at its point	Able to rotate on the spot. [SE3]
Robust Construction	Repeated tests	All wires and components are not loose and held firm at its position [SE4]
Intuitive User Interface	Operator control via webpage	Web interface that is easy to use [SE5]
Budget below £60	Compiled a BOM at the end of the report	Total cost is £57.49 Please refer to the cost analysis

Supplementing Evidences



Figure 7.4. Weight of rover.

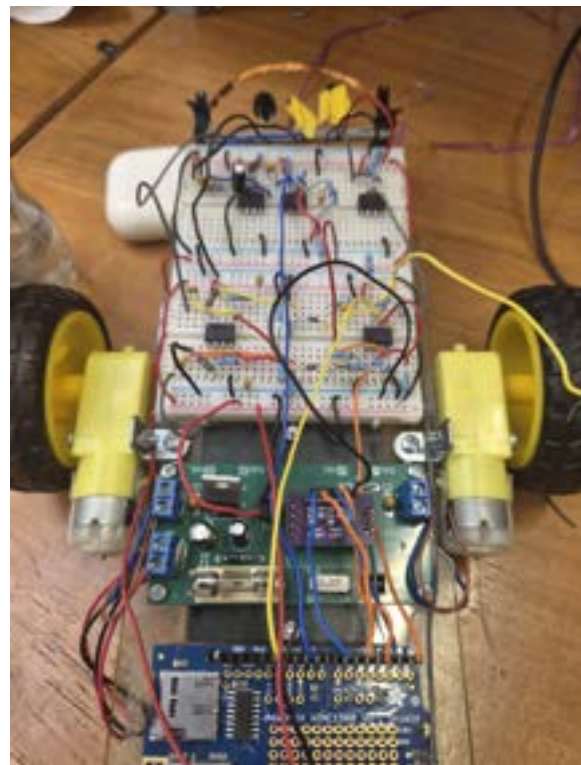


Figure 7.5. Robot can be controlled with minimal latency through small gap [Video on GitHub]

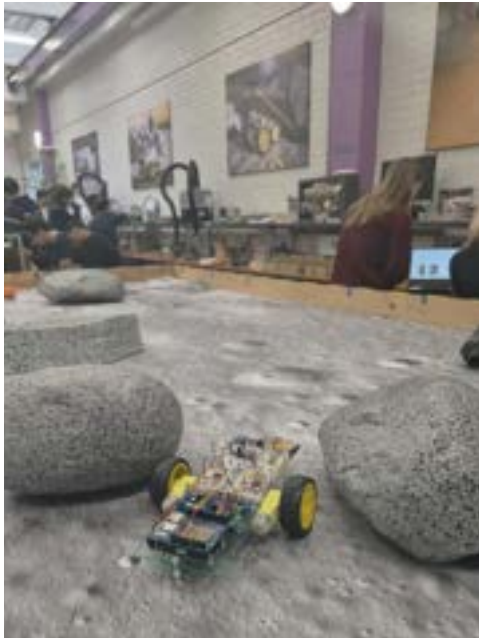


Figure 7.6. Robot can rotate around on the spot [Video on GitHub]

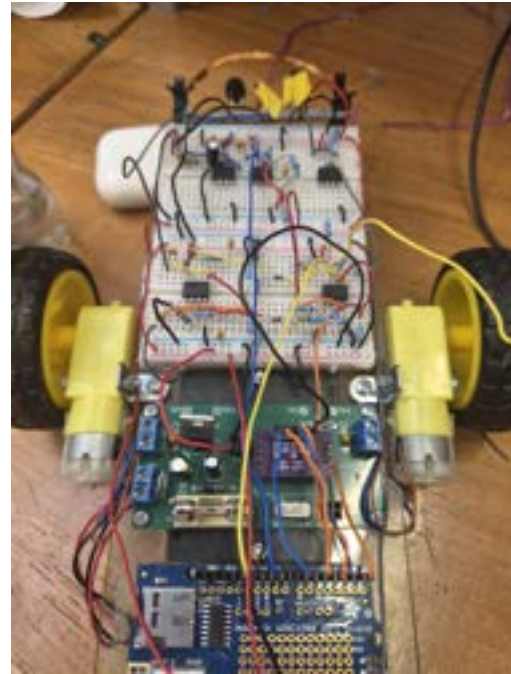


Figure 7.7 Components and wires are all in their right place



Figure 7.8. Web control interface of the rover.

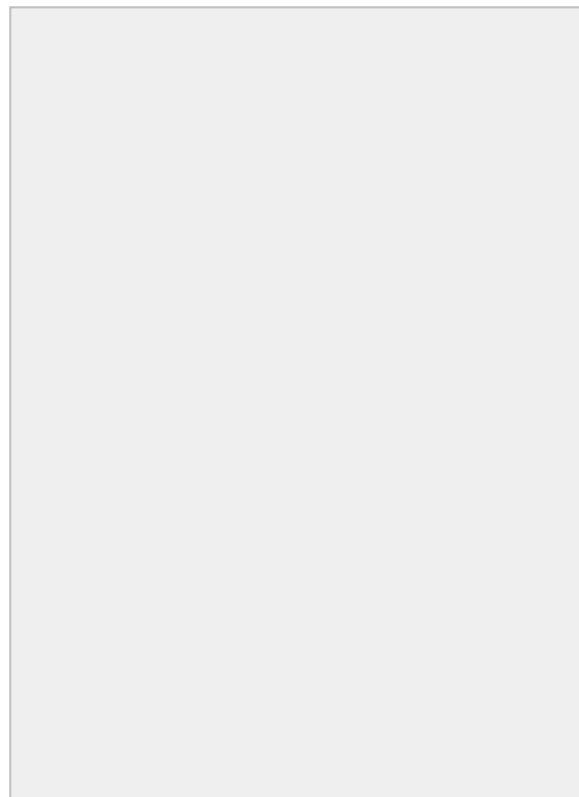


Figure 7.9. Final completed rover.

8. Conclusion

The EEE Lunar Rover designed by group 10 is a two-wheel drive rover that is controlled by the user and can successfully identify different rock types. It classifies all four rock types - Basaltoid, Gravion, Regolix and Lunarite - by combining infrared, ultrasound and magnetic measurements. The age of the rock can be detected using a radio signal detector. User interface capability is provided through the WINC1500 wifi shield that hosts a custom user interface webpage for user control of the rover. A low-latency, intuitive control system that allowed for precise navigation of the demo environment was achieved.

The rover maintained a final weight of 494.5g, significantly below the 750g limit, ensuring safe traversal over fragile lunar surfaces. Systematic testing demonstrated that the combination of infrared, ultrasound, magnetic, and radio subsystems provided 100% accuracy in classifying the four distinct rock types (Basaltoid, Gravion, Regolix, and Lunarite) by cross-referencing their unique signal profiles. Detector components were reused for different sensor subsystems (MCP 6022 I/P, LM393P, 1N4148) to ensure circuit efficiency and cost effectiveness.

The project timeline was tight and revealed possible expansions for future further development of the rover. Current circuitry was implemented using a breadboard. A PCB could be designed to improve circuit integrity and allow for more robust mounting. Additionally, custom chassis work could be followed up on to enhance the compactness and manoeuvrability of the rover. More complex 3D-printing and motor drive-train systems could be investigated.

Cost Analysis

Here is a bill of materials used by this project. (Excludes resistors, capacitors and other components provided at the start/in the lab {EEEBug Expansion Set,EEEBug, Metro M0}).

No.	Name & Pic & Quantity	Purpose	No.	Name & Pic & Quantity	Purpose
1	DRV5053RAQLPG  [1] Total Cost = 0.59	Used for Magnetic Subsystem	4	MCP6022-I/P  [3] Total cost = 4.41	Opamp used in all circuits,
2	400SR10B  [1] Total Cost = 7.76	Used for receiving ultrasonic signals for ultrasonic subsystems	5	LM393P  [2] Total cost = 0.92	Comparator used in all subsystems
3	TEFT4300  [1] Total Cost = 0.4	Used for infrared circuit.	6	1N4148  [2] Total Cost = 0.12	Fast-switching diode
	Total cost = 14.2				

First Name: Devesh
Last Name: Khemani
CID: 02723399
Group Number: 10

ELEC40006-Electronics Engineering Design Project 1 Reflection on Professional Practice Form

Please answer all of the following questions in a total of 300 words (+10%). This is a compulsory component of the module and must be passed.

Include your name, surname, and CID number on the form.

Each group member must complete the Reflection on Professional Practice Form individually.

All completed forms must be compiled and attached to the end of your group's project report. Submit the full report, including all reflection forms, by 17:00pm on 11 June 2026.

1. How effectively did your group function as a team? How effective was your personal contribution to the team?

My group functioned well as a team. We split the workload based on preference, dividing into two groups, one handling rock age detection and the other handling rock type classification and movement. We worked on our own subsystems individually, with weekly lab meetings for progress updates. My primary technical responsibility was the IR sensing subsystem, encompassing circuit design, component selection, and firmware development. Beyond this, I developed the non-technical sections of the interim presentation and authored the design process sections of the final report, including requirements specification, system architecture, and project planning. This cross-cutting role required me to maintain an understanding of all subsystems and ensured consistency across the team's outputs.

2. How did a diversity of people in the teams help you to perform better as a team?

The diversity of backgrounds and skill sets within the team meant that challenges were approached from multiple perspectives simultaneously. When technical issues arose, team members drew on their individual knowledge of hardware design and software implementation to propose varied solutions. By evaluating these collectively and selecting the best elements from each, the team arrived at more robust solutions than any individual could have reached independently. This was particularly valuable during integration, where diverse perspectives helped identify interface conflicts early.

3. What skills and knowledge did you acquire in this project? How?

I gained engineering knowledge such as hardware-software integration to manipulate the hardware into intended functionality. I also gained a basic understanding of some different sensors and how they work. However, I feel I have gained far more engineering skills, such as how to function as part of an interdisciplinary team, articulating complex technical concepts clearly to others, and learning to have objective, ego-free evaluation of the ideas and perspectives proposed. Additionally, I achieved a better understanding of how engineers use their technical skills to tackle real-world problems alongside a team with different technical skills.

First Name: Zifan
Last Name: Ye
CID: 02669492
Group Number: 10

ELEC40006-Electronics Engineering Design Project 1 Reflection on Professional Practice Form

Please answer all of the following questions in a total of 300 words (+10%). This is a compulsory component of the module and must be passed.

Include your name, surname, and CID number on the form.

Each group member must complete the Reflection on Professional Practice Form individually.

All completed forms must be compiled and attached to the end of your group's project report. Submit the full report, including all reflection forms, by 17:00pm on 11 June 2026.

1. How effectively did your group function as a team? How effective was your personal contribution to the team?

1. Overall, I think our group functioned pretty well together. Everyone was supportive, and we managed to get things done. Wangmo and Chris were especially responsible, and they helped me out a lot whenever I felt stuck. However, we were a bit shorthanded because Mohammed didn't come for the project, which made things a little more challenging for the rest of us. As for my personal contribution, I have to be honest that I might not have been at my absolute best. With the summer break coming up, my mind was already a bit on holiday mode, so my focus drifted a little. But I still tried my best to keep up and do my part to support the team.

2. How did a diversity of people in the teams help you to perform better as a team?

2. Working with different people really pushed me out of my comfort zone. I'm naturally an introverted person, so collaborating in this group was a big personal challenge for me. But because everyone had different personalities and ways of working, it forced me to talk with others. Watching how responsible and driven Chris and Wangmo were really motivated me to stay on track, even when I was feeling a bit distracted by the upcoming holidays. Their energy helped keep the team moving forward.

3. What skills and knowledge did you acquire in this project? How?

3. The biggest thing I learned was definitely communication and how to work closely with others. Since I used to be quiet, I improved a lot in terms of expressing my thoughts and just getting along in a team environment. On the technical side, my hands-on and practical skills have always been a bit weak. This project gave me a chance to build things and do some handworks in the lab, which definitely boosted my confidence and improved those practical skills a bit.

First Name: Albert
Last Name: Ma
CID: 02679083
Group Number: 10

ELEC40006-Electronics Engineering Design Project 1 Reflection on Professional Practice Form

Please answer all of the following questions in a total of 300 words (+10%). This is a compulsory component of the module and must be passed.

Include your name, surname, and CID number on the form.

Each group member must complete the Reflection on Professional Practice Form individually.

All completed forms must be compiled and attached to the end of your group's project report. Submit the full report, including all reflection forms, by 17:00pm on 11 June 2026.

1. How effectively did your group function as a team? How effective was your personal contribution to the team?

1. Response to Q#1: Overall, our group worked reasonably effectively as a team. We stayed in frequent contact through WhatsApp, sometimes discussing urgent problems late at night. For important decisions, we arranged in-person meetings, usually once or twice a week, alongside smaller working sessions. For example, I often met separately with the team leader two or three times a week to test code and circuits. My personal contribution was substantial. I was fully responsible for movement control and the user interface, and I also integrated data from different sensors into the web display. I participated actively in coding, debugging, integrating, testing, and refining the final rock-type output.

2. How did a diversity of people in the teams help you to perform better as a team?

2. Response to Q#2: The diversity of the team helped the project because members had different technical strengths, experiences, and problem-solving approaches. Some focused more on hardware, some on software, and others on organisation or sensor testing. This was useful during integration, where problems often involved both hardware and software. Although I mainly handled software integration and Chris focused more on hardware, we often gave each other useful suggestions. For instance, I noticed that some pin choices affected the accuracy of displayed values, while he pointed out that some conditions in my code were filtering out valid data. These exchanges helped me approach problems more broadly.

3. What skills and knowledge did you acquire in this project? How?

3. Response to Q#3: Through this project, I gained deeper knowledge of HTML-based coding, web interfaces, system debugging, and GitHub workflow. I learned how to integrate VS Code with GitHub, upload folders, use .gitignore to protect private files, and pull changes between my two computers. I also learned how to upload Arduino code through VS Code and used GitHub CLI for uploading and pulling without relying only on VS Code. More importantly, I learned that debugging requires confidence as well as patience. Several issues that initially seemed to be caused by my code were later found to be loose wires, hardware instability, or network delays in the EEE Rover. This taught me to trust my work while testing carefully.

First Name: Wangmo
Last Name: Koo
CID: 02577935
Group Number: 10

ELEC40006-Electronics Engineering Design Project 1 Reflection on Professional Practice Form

Please answer all of the following questions in a total of 300 words (+10%). This is a compulsory component of the module and must be passed.

Include your name, surname, and CID number on the form.

Each group member must complete the Reflection on Professional Practice Form individually.

All completed forms must be compiled and attached to the end of your group's project report. Submit the full report, including all reflection forms, by 17:00pm on 11 June 2026.

1. How effectively did your group function as a team? How effective was your personal contribution to the team?

As a team we effectively split one large objective into several specialised sub-objectives based on each member's individual expertise and preference. This approach allowed everyone to work in their strongest areas and deliver the best results for each specialised subsystem. For example, when I ran into difficulties assembling my circuit, my group manager provided guidance and walked me through the process, which let me focus my strengths on explaining and summarising our findings for the report. My own contributions were as follows: I attended the lab and meetings consistently, planned out the circuit for my subsystem (the ultrasonic detection circuit), and served as the group's parts approver. Beyond this, I helped write report sections outside my planned area of work. Where some documentation needed further development, I spent extra time — including overnight — studying and analysing the relevant circuits so that I could rewrite those sections and help ensure the whole report was accurate and well explained.

2. How did a diversity of people in the teams help you to perform better as a team?

The diversity of our group members allowed us to divide the work into smaller tasks and tackle them simultaneously. This sped up the development stage and brought efficiencies when working across different subsystems. Working alongside people with different backgrounds also exposed me to a range of problem-solving approaches, which strengthened our final design decisions.

3. What skills and knowledge did you acquire in this project? How?

I improved my circuit-building skills and, in particular, my simulation skills in LTspice. I learned how to effectively simulate a circuit from a photo taken by another teammate, which I then verified against physical measurements the next day. Finally, taking on extra documentation tasks allowed me to develop circuit-backtracking skills and the ability to analyse a previously unseen circuit by splitting it into smaller subcircuits, which often aided my understanding.

First Name: Christopher
Last Name: Koh
CID: 02589209
Group Number: 10

ELEC40006-Electronics Engineering Design Project 1 Reflection on Professional Practice Form

Please answer all of the following questions in a total of 300 words (+10%). This is a compulsory component of the module and must be passed.

Include your name, surname, and CID number on the form.

Each group member must complete the Reflection on Professional Practice Form individually.

All completed forms must be compiled and attached to the end of your group's project report. Submit the full report, including all reflection forms, by 17:00pm on 11 June 2026.

1. How effectively did your group function as a team? How effective was your personal contribution to the team?

Overall, the group functioned reasonably effectively, although communication was inconsistent during much of the project. Team members often worked independently on their allocated subsystems, making overall project progress difficult to track. However, coordination improved significantly towards the end of the project, allowing successful integration and completion of all major deliverables. My contribution focused on the design, prototyping and testing of several radio, magnetic, infrared and ultrasound subsystems, including circuit design and signal conditioning. I also contributed to system integration and mounting of sensors.

2. How did a diversity of people in the teams help you to perform better as a team?

The diversity of the team exposed me to different working styles, levels of experience and approaches to problem-solving. It was also beneficial to gain insight from team members with different areas of expertise, as this provided alternative perspectives on design decisions and technical challenges. While this occasionally created communication and coordination difficulties, it required me to be more flexible and adaptable in my approach. It also helped me develop skills in managing disagreements, working with difficult personalities and maintaining progress despite differing priorities. Overall, these experiences improved my teamwork and communication skills and better prepared me for collaborative engineering projects.

3. What skills and knowledge did you acquire in this project? How?

This project allowed me to develop both technical and professional skills. From a technical perspective, I improved my ability to research and evaluate different engineering solutions, including sensor technologies, communication systems and signal-conditioning circuits. I learned how to compare alternative approaches, justify design decisions and select components based on performance, cost and project constraints. The project also improved my project management skills. Working within a multidisciplinary engineering team taught me how to coordinate tasks, manage dependencies between subsystems and monitor project progress. In addition, I gained a better understanding of the engineering project workflow, from initial research and concept development through prototyping, testing, integration and final evaluation. These experiences provided valuable insight into how complex engineering projects are planned and executed.